

NAGARMITRA

Smart City Civic Issue Reporting and Distributed Service Management System

A Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

K. Saicharan - 2520030002
P. Harshith - 2520030393
K. Rishi Varma - 2520030248

Under the Guidance of:

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

CERTIFICATE



Department of Computer Science and Engineering

This is to certify that the project report entitled “Nagarmitra – Smart City Civic Issue Reporting and Distributed Service Management System” is a bonafide record of the work carried out by K. Saicharan (2520030002), P. Harshith (2520030393) and K. Rishi Varma (2520030248), in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

**Signature of
the HOD**

DECLARATION

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING



The Project Report entitled “**Smart City Civic Issue Reporting and Distributed Service Management System** ” is a record of Bonafide work of **KARAMSHETTI SAICHARAN - 2520030002 , P.HARSHITH - 2520030393 , K. RISHIVARAMA - 2520030248** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K. Saicharan
2520030002

P. Harshith
2520030393

K. Rishi Varma
2520030248

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr **Yerragudipadu Subbarayudu**, and my guide Dr **Yerragudipadu Subbarayudu**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. Saicharan
P. Harshith
K. Rishi Varma

ABSTRACT

Cities depend on local municipal bodies to look after roads, drainage, garbage collection, street lighting and water supply. When something goes wrong, citizens usually have to complain through phone calls, office visits or separate online forms. In many existing systems, these complaints are sorted by hand, sometimes sent to the wrong department, and rarely followed up with clear updates. Citizens often cannot tell whether their complaint has been seen, and a complaint may be marked closed without any proof that the work was done.

Nagarmitra is a web-based civic issue reporting and service management platform built to address these problems. A citizen can report a civic issue such as road damage, garbage, drainage, streetlight or water-related problems by entering a description, uploading images and marking the location of the problem. The complaint is then handled through a single workflow that connects four types of users: citizens, managers, field workers and administrators.

The complaint lifecycle in Nagarmitra begins when a citizen submits a complaint. A manager reviews it, classifies it and assigns it to a suitable department or field worker. The field worker sees the assigned task, updates its status as the work progresses and submits proof after the work is completed. The citizen can follow the progress of the complaint, receives notifications about changes, and can give feedback once the issue is resolved. The administrator manages the users and the overall working of the system.

The system is developed using React.js for the frontend, Python FastAPI for the backend and MySQL for the database. The frontend communicates with the backend through HTTP APIs, and the backend stores and retrieves complaint, user, assignment, notification and feedback data from MySQL. Map and location features are used to capture where an issue is present, and image upload is used to attach visual evidence to complaints and resolution proof. Role-based access limits each user to the screens and actions meant for their role.

For testing, the APIs are checked using Postman and the Swagger/OpenAPI documentation generated by FastAPI, while the frontend is checked for functionality, responsiveness and performance using browser tools and Lighthouse.

The main purpose of Nagarmitra is to make civic complaint handling more organized and transparent by keeping citizens, managers and field workers connected through one platform. Possible future improvements, such as a mobile application, automatic complaint classification, duplicate complaint detection and real-time notifications, are discussed separately as future enhancements and are not part of the current implementation.

Keywords: Civic issue reporting, complaint management, React.js, FastAPI, MySQL, role-based access, smart city.

TABLE OF CONTENTS

CERTIFICATE.....	i
DECLARATION.....	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
CHAPTER 1: INTRODUCTION.....	1
1.1 Background of the Project.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	1
1.4 Existing System.....	2
1.5 Problems in Existing Systems.....	2
1.6 Proposed System.....	3
1.7 Scope of the Project.....	4
1.8 Target Users.....	4
CHAPTER 2: SYSTEM REQUIREMENTS.....	5
2.1 Hardware Requirements.....	5
2.2 Software Requirements.....	5
2.3 Development Tools and Frameworks.....	5
CHAPTER 3: TECHNOLOGY STACK.....	7
3.1 Frontend — React.js.....	7
3.2 Backend — Python FastAPI.....	7
3.3 Database — MySQL.....	7
3.4 API Communication.....	8
3.5 Maps and Location Services.....	8
3.6 Image Upload / Storage.....	8
3.7 Other Technologies Actually Used.....	8
CHAPTER 4: SYSTEM ARCHITECTURE.....	10
4.1 High-Level Architecture.....	10
4.2 Frontend Layer.....	10
4.3 Backend Layer.....	11
4.4 Database Layer.....	11
4.5 Frontend–Backend Communication.....	11
4.6 Backend–Database Communication.....	11
4.7 Overall System Workflow.....	11
CHAPTER 5: SYSTEM DESIGN.....	13
5.1 Use Case Diagram.....	13
5.2 Actors and Their Responsibilities.....	13
5.3 Complaint Lifecycle.....	13
5.4 System Flow.....	14
5.5 ER Diagram.....	15
5.6 Database Schema.....	15
5.7 Role-Based Access Design.....	16

CHAPTER 6: MODULE-WISE IMPLEMENTATION.....	18
6.1 User Authentication Module.....	18
6.2 Citizen Management Module.....	18
6.3 Complaint Management Module.....	18
6.4 Manager/Triage Module.....	19
6.5 Field Operations Module.....	19
6.6 Location and Map Module.....	19
6.7 Notification Module.....	20
6.8 Feedback Module.....	20
6.9 Admin Module.....	20
6.10 Database Management Module.....	21
CHAPTER 7: SYSTEM FEATURES.....	22
7.1 User Registration and Login.....	23
7.2 Role-Based Navigation.....	23
7.3 Civic Issue Reporting.....	24
7.4 Image Upload.....	24
7.5 Location-Based Reporting.....	25
7.6 Complaint Tracking.....	25
7.7 Complaint Assignment.....	25
7.8 Field Worker Updates.....	26
7.9 Resolution Proof.....	26
7.10 Notifications.....	27
7.11 Feedback.....	27
7.12 Administrative Management.....	27
CHAPTER 8: TESTING.....	29
8.1 Frontend Testing.....	29
8.2 Responsive Testing.....	29
8.3 API Testing Using Postman.....	30
8.4 Swagger/API Documentation Testing.....	31
8.5 Database Testing.....	31
8.6 Frontend–Backend Integration Testing.....	32
8.7 Authentication and Authorization Testing.....	32
8.8 Lighthouse / Performance Testing.....	32
8.9 Test Cases and Results.....	33
CHAPTER 9: DEPLOYMENT AND SETUP.....	36
9.1 Development Environment.....	36
9.2 Frontend Setup.....	36
9.3 Backend Setup.....	36
9.4 MySQL Database Setup.....	37
9.5 Environment Variables.....	37
9.6 Running the Complete Application.....	37
9.7 API Server.....	37
9.8 Frontend–Backend Connection.....	38
CHAPTER 10: CHALLENGES AND LIMITATIONS.....	39
10.1 Challenges Faced During Development.....	39

10.2 Frontend–Backend Integration Challenges.....	39
10.3 Database Integration Challenges.....	39
10.4 Role-Based Access Challenges.....	39
10.5 Responsive Design Challenges.....	39
10.6 API Testing Challenges.....	40
10.7 Current Limitations.....	40
CHAPTER 11: FUTURE ENHANCEMENTS.....	41
11.1 Mobile Application.....	41
11.2 Advanced AI-Based Complaint Classification.....	41
11.3 Duplicate Complaint Detection.....	41
11.4 Real-Time Notifications.....	41
11.5 Advanced Analytics.....	41
11.6 Cloud Deployment.....	41
11.7 Smart City Service Integration.....	41
11.8 Improved Location-Based Services.....	42
CHAPTER 12: CONCLUSION.....	43
CHAPTER 13: REFERENCES.....	44
APPENDIX A: APPLICATION SCREENSHOTS.....	45
APPENDIX B: API TESTING SCREENSHOTS.....	48
APPENDIX C: DATABASE SCREENSHOTS.....	49
APPENDIX D: RESPONSIVE TESTING SCREENSHOTS.....	50
APPENDIX E: IMPORTANT CONFIGURATION / API EXAMPLES.....	51

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Local governments are responsible for basic services that people use every day, such as road maintenance, waste collection, drainage, street lighting and water supply. As cities grow, the number of small problems in these services also grows. A damaged road, an overflowing drain or a streetlight that does not work may look minor, but such problems affect safety, health and daily travel.

Citizens usually report these problems through phone calls, written applications, visits to municipal offices or general-purpose complaint portals. Once a complaint is submitted, the citizen often has little information about what happens next. Municipal staff, on the other hand, may receive complaints in different formats and need to sort and forward them manually to the right department.

Nagarmitra is a web-based platform developed to bring citizens, municipal managers and field workers onto a single system. It allows civic complaints to be reported with descriptions, images and location details, and it supports the steps that follow: reviewing, assigning, working on and closing the complaint.

1.2 Problem Statement

Existing civic complaint handling methods depend heavily on manual work and separate systems. Complaints are classified and forwarded by hand, which can lead to delays and wrong routing. Citizens do not get clear status updates, field workers are often not connected to the same system as the office staff, and complaints can be closed without evidence that the work was completed. There is a need for a single web-based system that manages a complaint from the moment it is reported until the moment it is resolved and reviewed by the citizen.

1.3 Objectives

The main objectives of Nagarmitra are:

- To provide a web-based platform where citizens can report civic issues with a description, images and location.
- To allow managers to review, classify and assign complaints to suitable departments or field workers.

- To give field workers a way to view assigned tasks, update work status and submit resolution proof.
- To let citizens track the progress of their complaints and receive notifications about updates.
- To collect citizen feedback after a complaint is resolved.
- To provide role-based access for citizens, managers, field workers and administrators.
- To store all complaint-related data in a structured MySQL database.

1.4 Existing System

In the existing approach, a citizen reports a civic problem by calling a helpline, visiting an office, writing an application or using a basic complaint form on a website. The complaint is received by an office, where an employee reads it, decides which department it belongs to and forwards it. The department then informs a field worker, usually through phone calls or paper-based instructions. When the work is done, the complaint is updated or closed by the office.

Some municipal websites and applications allow online complaint submission, but most of them only collect the complaint and do not manage the full process after that. The citizen, the office staff and the field worker generally use different tools, or no common tool at all.

1.5 Problems in Existing Systems

The following problems were identified in the existing way of handling civic complaints.

No.	Problem	Description
1	Manual Triage and Misrouting Errors	Complaints are read and classified manually by office staff. A complaint may be sent to the wrong department, which causes delay because it has to be forwarded again.
2	Lack of Transparency	After submitting a complaint, the citizen usually cannot see whether it has been reviewed, assigned or worked on.
3	Disconnected Field-Worker Loop	Field workers are not part of the same system as the office. Task instructions and work updates are passed through calls or paper, so progress is not recorded properly.

No.	Problem	Description
4	Administrative Closure Without Proof of Work	A complaint can be marked as closed from the office without any photo or record showing that the work was actually completed.
5	Architectural and Database Limitations	Many existing portals are built as a single simple application with limited data storage. They do not keep proper relations between citizens, complaints, departments, workers and status history, which makes tracking and reporting difficult.
6	Siloed, Single-Purpose Portals	Different portals exist for different purposes, for example one only for filing complaints. They do not share data with each other, so the complaint process is broken into disconnected parts.

1.6 Proposed System

Nagarmitra is proposed as a single platform that covers the complete life of a civic complaint. A citizen reports an issue with a description, images and a map location. A manager reviews the complaint, classifies it and assigns it to a suitable department or field worker. The field worker views the task, updates its status and submits resolution proof. The citizen can track the complaint, receive notifications and give feedback after resolution.

The word “distributed” in the project title refers to the way complaint handling responsibilities are distributed among managers, departments and field workers inside one platform. Each role has its own dashboard and actions, and the backend keeps them connected through a common database.

The proposed system addresses the problems listed in Section 1.5 as follows:

Problem in Existing System	How Nagarmitra Addresses It
Manual triage and misrouting	Managers review and classify complaints inside the system and assign them to a department or field worker.
Lack of transparency	Citizens can track complaint status and receive notifications on updates.
Disconnected field-worker loop	Field workers log in to their own dashboard, see assigned tasks and update status directly.
Closure without proof of work	Field workers submit resolution proof (for example, images) as part of completing a task.

Problem in Existing System	How Nagarmitra Addresses It
Architectural and database limitations	A separate React frontend, FastAPI backend and MySQL relational database keep the data structured and the layers independent.
Siloed, single-purpose portals	Citizens, managers, field workers and administrators use one platform with role-based access.

1.7 Scope of the Project

The current scope of Nagarmitra includes:

- User registration, login and role-based access for citizens, managers, field workers and administrators.
- Reporting of civic issues such as road damage, garbage, drainage, streetlight problems, water-related issues and other local complaints.
- Image upload and location selection while reporting a complaint.
- Complaint review, classification and assignment by managers.
- Task viewing, status updates and resolution proof submission by field workers.
- Complaint tracking, notifications and feedback for citizens.
- Administrative management of users and system data.

The following items are not part of the current implementation and are discussed only as future enhancements in Chapter 11: a mobile application, automatic AI-based complaint classification, duplicate complaint detection, real-time notifications, advanced analytics, cloud deployment and integration with other smart city services.

1.8 Target Users

User	Description
Citizen	Residents who report civic issues, track their complaints and give feedback.
Manager / Department Manager	Municipal staff who review, classify and assign complaints and monitor their progress.
Field Worker	Staff who carry out the assigned work on site and submit status updates and resolution proof.
Administrator	User who manages the accounts and overall data of the system.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware listed below is what is needed to develop and run Nagarmitra on a single computer. Users of the system only need a device with a modern web browser.

Component	Requirement
Processor	Intel Core i5 / AMD Ryzen 3 or higher
RAM	8 GB or more recommended (4 GB minimum)
Storage	At least 5 GB of free disk space for tools, project files and database
Display	Standard monitor or laptop screen
Network	Internet connection for installing packages and loading map services

2.2 Software Requirements

Software	Purpose
Operating System (Windows / Linux / macOS)	Platform for development and execution
Web browser (for example, Google Chrome)	Running the frontend and browser developer tools
Node.js and npm	Installing and running the React.js frontend
Python 3.x	Running the FastAPI backend
MySQL Server	Database for the application
Postman	API testing
Git	Version control

2.3 Development Tools and Frameworks

Tool / Framework	Use in Nagarmitra
Google Antigravity IDE	Development environment used for writing and editing the code
React.js	Building the user interface

Tool / Framework	Use in Nagarmitra
Python FastAPI	Building the backend APIs
MySQL	Storing application data
Postman	Sending requests to the APIs and checking responses
Swagger / OpenAPI	API documentation and testing interface provided with FastAPI
Git / GitHub	Version control and code sharing among team members

CHAPTER 3: TECHNOLOGY STACK

This chapter describes the technologies used in Nagarmitra and the reason each one was chosen.

Layer	Technology
Frontend	React.js
Backend	Python FastAPI
Database	MySQL
API testing	Postman; Swagger UI (OpenAPI)
Development environment	Google Antigravity IDE
Version control	Git / GitHub

3.1 Frontend — React.js

The user interface of Nagarmitra is built with React.js, a JavaScript library for building interfaces from reusable components. Screens such as the login page, complaint form, dashboards and notification panel are built as separate components, which makes the code easier to manage and reuse. React also keeps the page updated when the data changes, so the user does not have to reload the page after every action. Different dashboards are shown to citizens, managers, field workers and administrators based on the role of the logged-in user.

3.2 Backend — Python FastAPI

The backend of Nagarmitra is developed using FastAPI, a Python framework for building web APIs. It handles requests from the frontend, applies the rules of the application, such as who is allowed to perform which action, and communicates with the database. FastAPI validates incoming data using Pydantic models and automatically generates interactive API documentation based on the OpenAPI standard, which was useful for testing the APIs during development.

3.3 Database — MySQL

MySQL is a relational database, so data is stored in tables and the relations between them are defined using keys. This suits Nagarmitra because complaints are connected to

citizens, departments, field workers, status updates, notifications and feedback. Using a relational database helps keep the data consistent and makes it possible to fetch related information, for example all complaints assigned to a particular field worker.

3.4 API Communication

The React frontend and the FastAPI backend communicate over HTTP. The frontend sends requests to the backend endpoints, and the backend returns responses in JSON format. Actions such as logging in, submitting a complaint, assigning a task or updating status are each handled through separate API endpoints. Access to protected endpoints depends on the role of the user.

3.5 Maps and Location Services

Nagarmitra includes map and location functionality so that the exact place of a civic issue can be recorded with the complaint. This helps managers understand where the problem is and helps field workers reach the correct place. The location information is stored along with the complaint in the database.

3.6 Image Upload / Storage

Citizens can attach images while reporting a complaint, and field workers can attach images as resolution proof. The images are uploaded through the frontend to the backend, and the details needed to retrieve them are linked to the corresponding complaint record.

3.7 Other Technologies Actually Used

Technology	Use
Postman	Testing the backend APIs by sending requests and checking responses
Swagger UI / OpenAPI	Viewing and trying the APIs from the documentation page generated by FastAPI
Git / GitHub	Keeping track of code changes and sharing code among team members
Lighthouse (Chrome DevTools)	Checking performance and related quality aspects of the frontend
Google Antigravity IDE	Development environment

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture

Nagarmitra follows a three-layer architecture made up of a frontend layer, a backend layer and a database layer. The React frontend is what the user sees and uses. It sends requests to the FastAPI backend, which applies the application logic and reads or writes data in the MySQL database. Map and location services are used from the frontend to capture and show the location of complaints, and uploaded images are passed to the backend along with the complaint data.

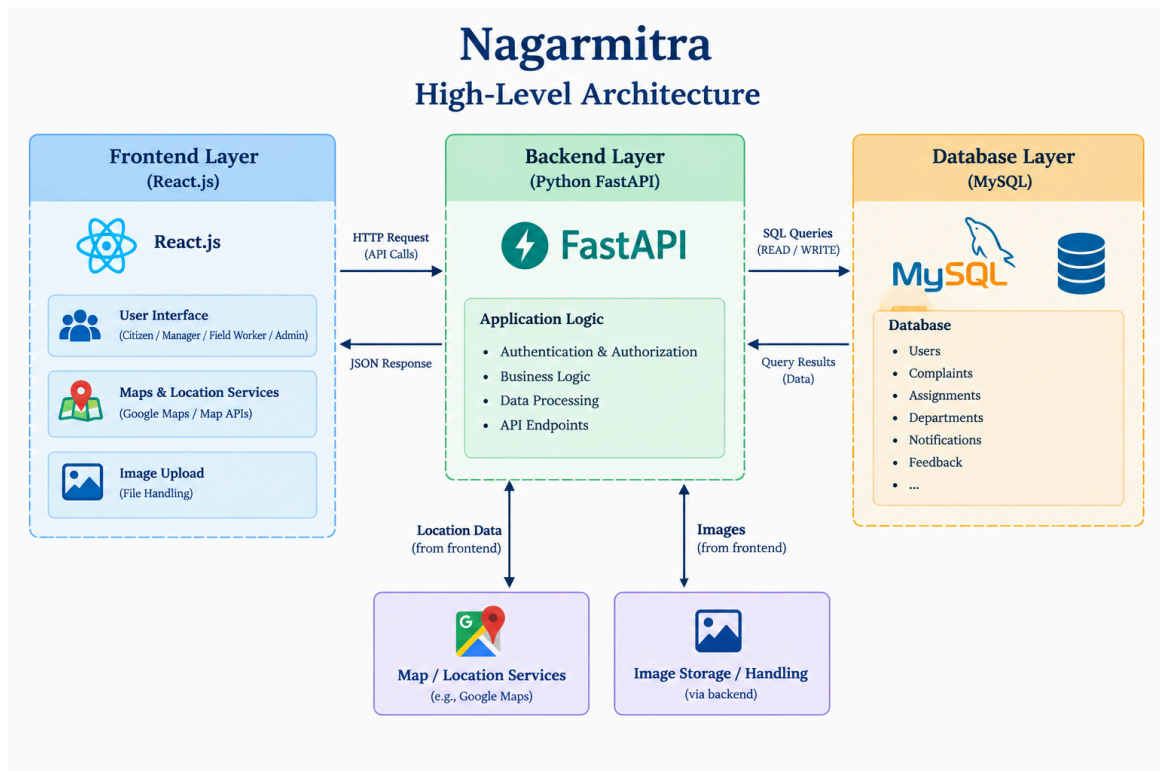


Figure 4.1: Nagarmitra system architecture

Layer	Technology	Main Responsibility
Frontend	React.js	User interface, forms, dashboards, role-based navigation
Backend	Python FastAPI	APIs, validation, authentication, role checks, business logic
Database	MySQL	Persistent storage of users, complaints, assignments, notifications and feedback

4.2 Frontend Layer

The frontend layer is a React application that runs in the browser. It contains the pages and components for registration, login, complaint reporting, complaint tracking, manager and field worker dashboards, notifications, feedback and administration. After login, the application shows the navigation and dashboard that belong to the user's role. The frontend does not store application data permanently; it requests data from the backend whenever it is needed.

4.3 Backend Layer

The backend layer is a FastAPI application that exposes the APIs used by the frontend. It receives requests, checks the input data, verifies who the user is and whether the user's role is allowed to perform the action, and then carries out the operation. Typical operations include creating a complaint, assigning it to a field worker, updating the status of a task, saving resolution proof, creating a notification and saving feedback.

4.4 Database Layer

The database layer is a MySQL database that stores the permanent data of the system. It contains the details of users and their roles, complaints with their location and images, departments, assignments, status updates, notifications and feedback. Relations between these tables are maintained using primary keys and foreign keys.

4.5 Frontend–Backend Communication

The frontend calls the backend APIs over HTTP. The request contains the data entered by the user, for example the login details or complaint form fields, and the backend replies with a JSON response. Image uploads are sent to the backend along with the complaint details. If a request is not valid or the user is not allowed to perform it, the backend returns an error response, which the frontend shows as a message to the user.

4.6 Backend–Database Communication

The backend is the only layer that communicates directly with the MySQL database. When a request needs data, the backend reads it from the database; when a request changes data, the backend writes the change. The frontend never connects to the database directly, which keeps the database credentials on the server side.

4.7 Overall System Workflow

1. The citizen registers or logs in and opens the complaint reporting page.
2. The citizen enters the description, uploads images and selects the location, and submits the complaint.
3. The backend validates the request and stores the complaint in the database.
4. The manager sees the new complaint, reviews and classifies it, and assigns it to a department or field worker.
5. The field worker sees the assigned task, updates its status and submits resolution proof after finishing the work.
6. Notifications are created for the users concerned when the complaint status changes.
7. The citizen tracks the complaint, sees that it is resolved and submits feedback.

CHAPTER 5: SYSTEM DESIGN

5.1 Use Case Diagram

The use case diagram shows how the four actors interact with Nagarmitra.

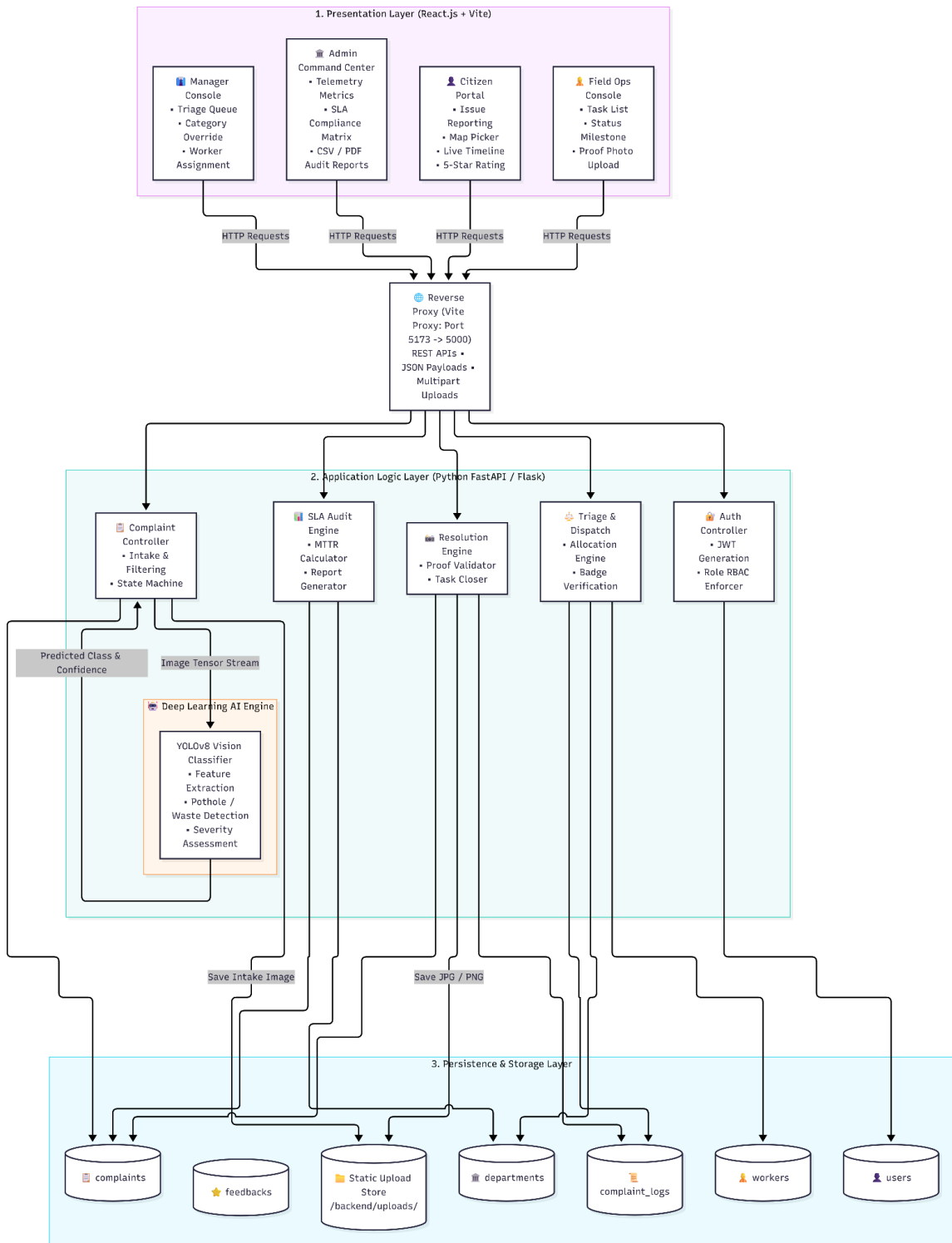


Figure 5.1: Use case diagram of Nagarmitra

5.2 Actors and Their Responsibilities

Actor	Responsibilities
Citizen	Register and log in; report civic issues with description, images and location; track complaints; view notifications; give feedback after resolution.
Manager / Department Manager	Review incoming complaints; classify them; assign them to a suitable department or field worker; monitor progress.
Field Worker	View assigned tasks; update task status; submit resolution proof after completing the work.
Administrator	Manage users and overall system data; oversee the working of the platform.

5.3 Complaint Lifecycle

A complaint passes through the following stages from the time it is reported until it is reviewed by the citizen.

Stage	Description	Performed By
1. Submitted	The citizen reports the issue with description, images and location.	Citizen
2. Reviewed / Classified	The manager checks the complaint and classifies it according to the type of issue.	Manager
3. Assigned	The complaint is assigned to a suitable department or field worker.	Manager
4. In Progress	The field worker works on the task and updates the status.	Field Worker
5. Resolved	The field worker completes the work and submits resolution proof.	Field Worker
6. Feedback	The citizen sees the resolved complaint and gives feedback.	Citizen

5.4 System Flow

The system flow starts with the citizen submitting a complaint and ends with the feedback given after resolution. At each step, the data is stored in the database and notifications are

generated for the users concerned. Closing a complaint depends on the resolution proof submitted by the field worker.

5.5 ER Diagram

The ER diagram shows the main entities of the database and the relations between them. In general, one citizen can submit many complaints, a complaint can be assigned to a field worker and belongs to a department, a complaint can have many status updates, and a resolved complaint can have feedback from the citizen. Notifications are linked to the user who receives them.

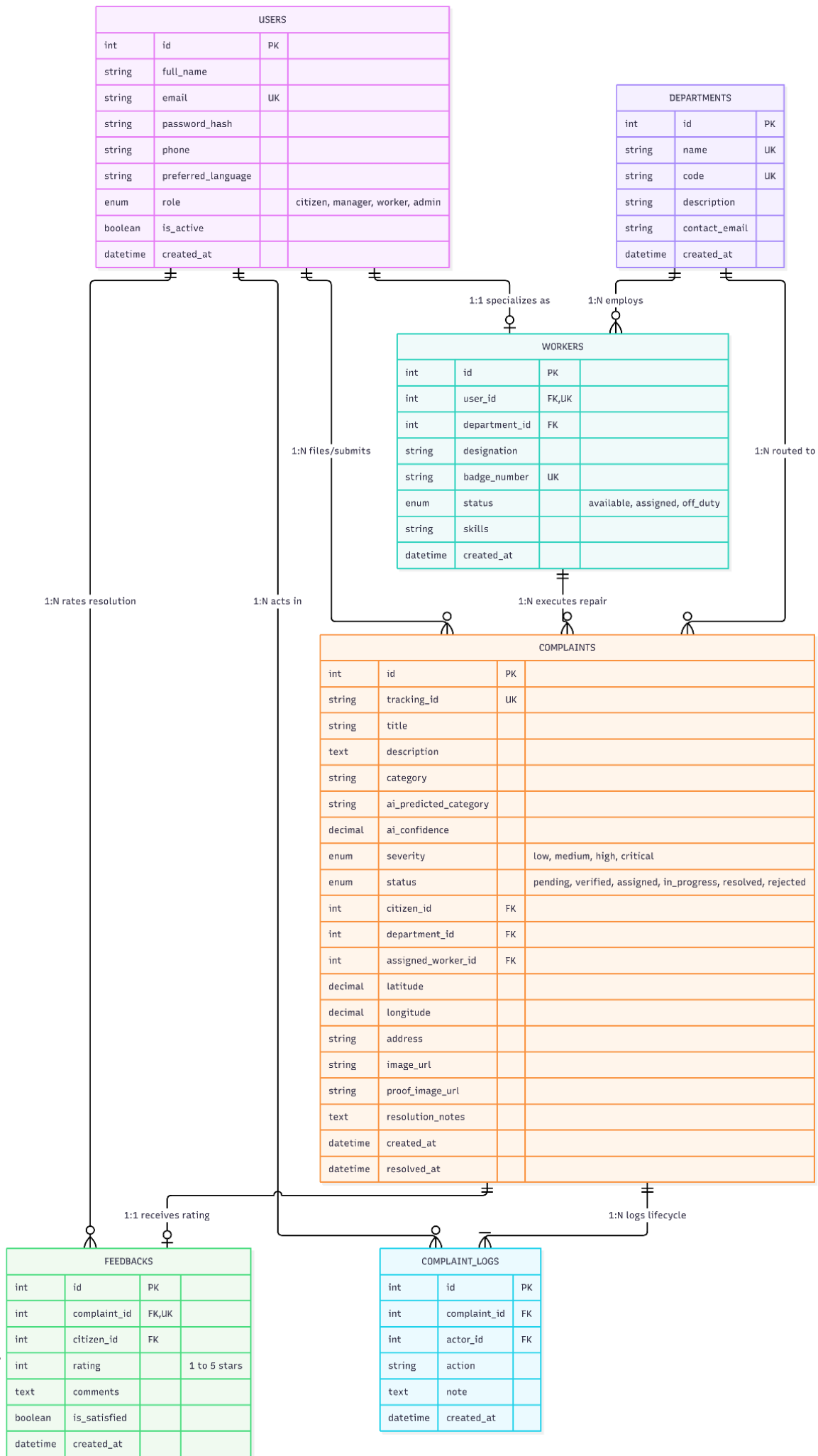


Figure 5.3: ER diagram of Nagarmitra

5.6 Database Schema

The main data stored by Nagarmitra is grouped into the logical entities given below. The table and column names used in the actual MySQL database are shown in the schema figure.

Entity	Purpose
Users	Stores login details and the role of each user (citizen, manager, field worker, administrator).
Complaints	Stores the description, issue type, location, images, current status and date of each complaint.
Departments	Stores the departments to which complaints can be assigned.
Assignments	Stores which manager assigned which complaint to which department or field worker.
Status updates / Resolution proof	Stores the progress updates and proof submitted by field workers.
Notifications	Stores messages generated for users when a complaint changes.
Feedback	Stores the citizen's feedback for a resolved complaint.

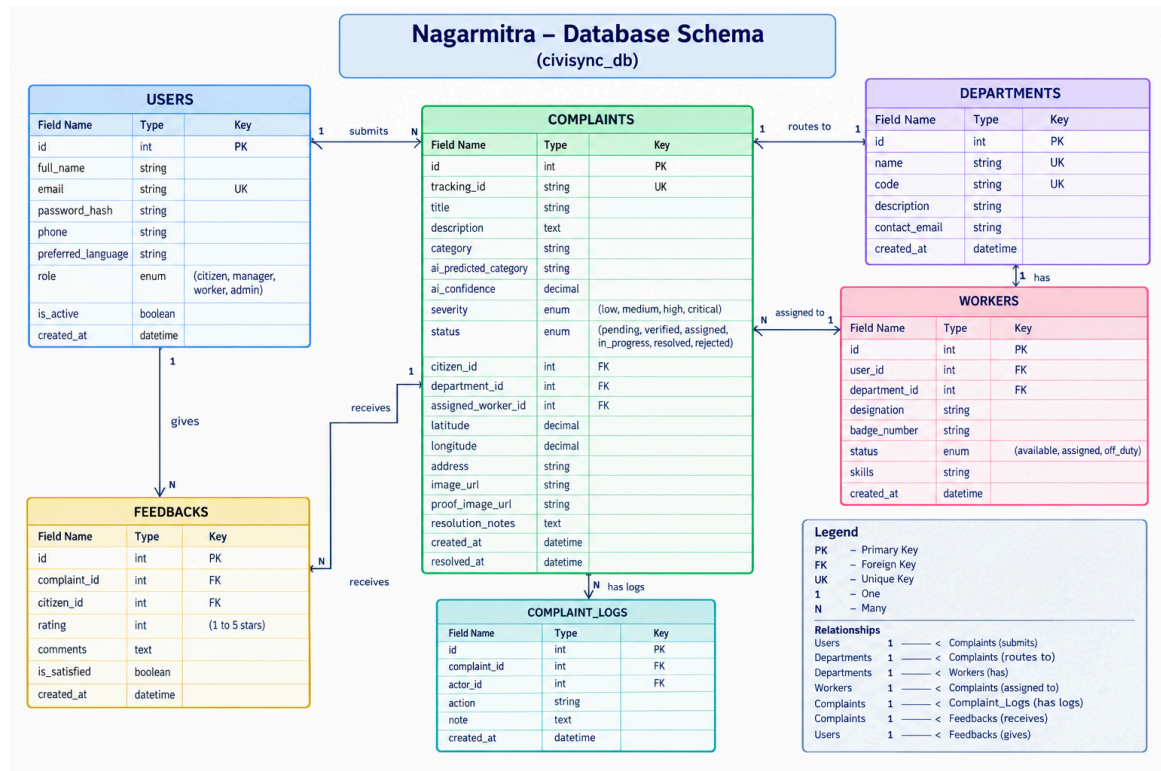


Figure 5.4: Database schema of Nagarmitra

5.7 Role-Based Access Design

Every user account has a role, and the role decides which screens the user can open and which APIs the user can use. The backend checks the role for each protected request, so access is controlled on the server side and not only by hiding options in the frontend.

Function	Citizen	Manager	Field Worker	Administrator
Register / log in	Yes	Yes	Yes	Yes
Report a complaint	Yes	No	No	No
Track own complaints	Yes	No	No	No
Review and classify complaints	No	Yes	No	No
Assign complaints	No	Yes	No	No
View assigned tasks	No	No	Yes	No
Update status and submit resolution proof	No	No	Yes	No
Give feedback	Yes	No	No	No
Manage users / system data	No	No	No	Yes

CHAPTER 6: MODULE-WISE IMPLEMENTATION

Nagarmitra is divided into ten modules. Each module is described below in terms of its purpose, main functions, working, the user involved and a short implementation note.

6.1 User Authentication Module

Purpose	To register users and allow only valid users to enter the system.
Main functions	User registration; login; logout; identifying the role of the user after login.
How it works	The user enters the details on the registration or login page. The backend checks them against the user records in the database and, if they are valid, allows access. The role of the user decides which dashboard is opened.
User / role	All users
Implementation	React forms send the entered data to FastAPI endpoints. The backend validates the input, checks the stored user record in MySQL and returns the result to the frontend.

6.2 Citizen Management Module

Purpose	To manage the citizen's account and give the citizen a place to see their own activity.
Main functions	View and manage own profile; view own complaints; open the citizen dashboard.
How it works	After login, the citizen dashboard loads the complaints submitted by that citizen and shows their current status.
User / role	Citizen
Implementation	The frontend requests the data of the logged-in citizen from the backend, which fetches it from the users and complaints tables.

6.3 Complaint Management Module

Purpose	To let citizens report civic issues and keep a record of each complaint.
Main functions	Create a complaint; enter description and issue type; upload images; attach location; view complaint details and status.
How it works	The citizen fills in the complaint form and submits it. The complaint is saved with the initial status and becomes visible to managers for review.

User / role	Citizen (creates and views); Manager (reviews)
Implementation	The form data, images and location are sent to a FastAPI endpoint, which validates them and stores the complaint in MySQL.

6.4 Manager/Triage Module

Purpose	To replace manual sorting of complaints with a review and assignment process inside the system.
Main functions	View new complaints; review details, images and location; classify the complaint; assign to a department or field worker; monitor progress.
How it works	The manager opens the manager dashboard, selects a complaint, checks the details and assigns it. The complaint status changes and the assigned worker can see the task.
User / role	Manager / Department Manager
Implementation	Manager-only API endpoints read the pending complaints and save the classification and assignment in the database. The backend checks that the user has the manager role.

6.5 Field Operations Module

Purpose	To connect field workers to the complaint system so that work progress is recorded.
Main functions	View assigned tasks; see complaint details and location; update task status; submit resolution proof.
How it works	The field worker logs in, sees the tasks assigned to them and updates the status as the work moves forward. After completing the work, the worker submits proof, such as an image, and the complaint is marked as resolved.
User / role	Field Worker
Implementation	Field worker endpoints return only the tasks assigned to that worker and accept status updates and proof uploads, which are stored against the complaint in the database.

6.6 Location and Map Module

Purpose	To record and display the place where a civic issue is present.
Main functions	Select or capture location while reporting; show the complaint location on a map to managers and field workers.

How it works	The citizen marks the location while creating the complaint. The saved location is later displayed on the map wherever the complaint details are shown.
User / role	Citizen (provides location); Manager and Field Worker (view location)
Implementation	The map component in the React frontend captures the location and sends it with the complaint data.

6.7 Notification Module

Purpose	To keep users informed about important changes.
Main functions	Create notifications on events such as assignment and status change; show notifications to the user.
How it works	When a complaint is assigned or its status changes, a notification record is created for the user concerned. The user can see it in the notification panel of the application.
User / role	Citizen, Manager, Field Worker
Implementation	The backend saves notification records in MySQL when the related events occur, and the frontend fetches and displays them.

6.8 Feedback Module

Purpose	To collect the citizen's opinion after a complaint is resolved.
Main functions	Submit feedback for a resolved complaint; store feedback for later viewing.
How it works	When a complaint is resolved, the citizen can open it and submit feedback. The feedback is linked to that complaint.
User / role	Citizen (submits); Manager / Administrator (view)
Implementation	A feedback form sends the entered data to a FastAPI endpoint, which stores it in the feedback table with a reference to the complaint.

6.9 Admin Module

Purpose	To manage the users and data of the whole system.
Main functions	Manage user accounts and roles; view overall system data; oversee complaints and departments.

How it works	The administrator logs in and uses the admin dashboard to view and manage the information available in the system.
User / role	Administrator
Implementation	Admin-only API endpoints read and modify the related data in MySQL after the backend confirms the administrator role.

6.10 Database Management Module

Purpose	To store all application data in an organised way.
Main functions	Store and retrieve user, complaint, assignment, status, notification and feedback data; maintain relations between tables.
How it works	Every module of the backend uses the database for reading and saving its data. The tables are linked using primary and foreign keys.
User / role	Used indirectly by all roles through the backend
Implementation	The MySQL database is accessed only by the FastAPI backend.

CHAPTER 7: SYSTEM FEATURES

This chapter describes the main features of Nagarmitra from the user’s point of view. Screenshot placeholders are given for each feature and are to be replaced with the actual screens of the application.

Table 7.1: Feature summary and status

Feature	Module	Implementation
7.1 User registration and login	User Authentication	Implemented
7.2 Role-based navigation	User Authentication / Admin	Implemented
7.3 Civic issue reporting	Complaint Management	Implemented
7.4 Image upload	Complaint Management	Implemented
7.5 Location-based reporting	Location and Map	Implemented
7.6 Complaint tracking	Citizen Management	Implemented
7.7 Complaint assignment	Manager/Triage	Implemented
7.8 Field worker updates	Field Operations	Implemented
7.9 Resolution proof	Field Operations	Implemented
7.10 Notifications	Notification	Implemented
7.11 Feedback	Feedback	Implemented
7.12 Administrative management	Admin	Implemented

In this report, “Implemented” means that the feature is part of the current application. “Tested” results are recorded only in Chapter 8. Features that are only planned are listed in Chapter 11.

7.1 User Registration and Login

New users create an account by entering the required details on the registration page. Registered users log in with their credentials. Access to the application is given only after the details are verified by the backend.


Nagarmitra
[ABOUT](#)
 Stronger Cities. Happier Communities.

[Home](#)
[Report Issue](#)
[Track Complaint](#)
[How It Works](#)
[Login](#)



Clean Environment & Civic Redressal

A public welfare portal connecting citizens with municipal health and environmental teams for cleaner, sustainable neighborhoods.

- 1. Report Civic & Sanitation Issues**
 Citizens submit complaints with photographic evidence for road hazards, water contamination, uncollected garbage, or public lighting.
- 2. Automated Triage & Department Routing**
 AI vision models evaluate environmental and safety urgency, routing tickets to the responsible public health supervisor.
- 3. Ground Resolution & Verification**
 Municipal teams complete repairs and upload photographic proof-of-work before grievance resolution is confirmed.

State Municipal Welfare Board

Toll Free: 1800-11-2026

STEP 2 OF 2 • ACCOUNT DETAILS

Create an Account

Register as a citizen or departmental official on the Nagarmitra municipal network.

Suitable Language: English (English)
 [Change Language](#)

Official Role Separation & Registration Advisory

- Citizen Registration (Recommended):** Select **Citizen** if you are a local resident to file grievances (potholes, garbage, water issues) and track updates.
- Permanent Role Lock:** Accounts registered as a **Citizen** cannot be signed in as other roles (Field Worker, Dept Manager, or Admin).
- Municipal Operatives:** Field Worker and Department Manager registrations require authorized municipal employee credentials.

Full Name *

Email Address *

Password *

Account Role / Designation *

☒ I agree to the municipal citizen charter, terms of public service, and privacy guidelines.

[Back](#)
[Create Account](#)

Already registered? [Sign In to your account](#)

Figure 7.1: Registration Page



Clean Environment & Civic Redressal

A public welfare portal connecting citizens with municipal health and environmental teams for cleaner, sustainable neighborhoods.

1

1. Report Civic & Sanitation Issues

Citizens submit complaints with photographic evidence for road hazards, water contamination, uncollected garbage, or public lighting.

2

2. Automated Triage & Department Routing

AI vision models evaluate environmental and safety urgency, routing tickets to the responsible public health supervisor.

3

3. Ground Resolution & Verification

Municipal teams complete repairs and upload photographic proof-of-work before grievance resolution is confirmed.

Sign In to Your Account

Access the official municipal grievance redressal and departmental portal.

Official Role Separation & Advisory

- Citizen Role (Recommended):** For public residents reporting local issues (potholes, garbage, water leaks) and tracking status.
- Security Rule:** If an account is registered as a **Citizen**, it cannot be signed in as other roles (Field Worker, Dept Manager, or Admin).
- Municipal Operatives:** Field Worker and Manager access requires authorized municipal staff credentials.

Email Address *

s@gmail.com

Password *

.....

Portal Access Role

Citizen (File & Track Civic Grievances) ▾

Sign In as Citizen

Figure 7.2: Login Page

7.2 Role-Based Navigation

After login, the user is taken to the dashboard that belongs to their role. Citizens, managers, field workers and administrators see different menus and options, so each user sees only the actions meant for them.

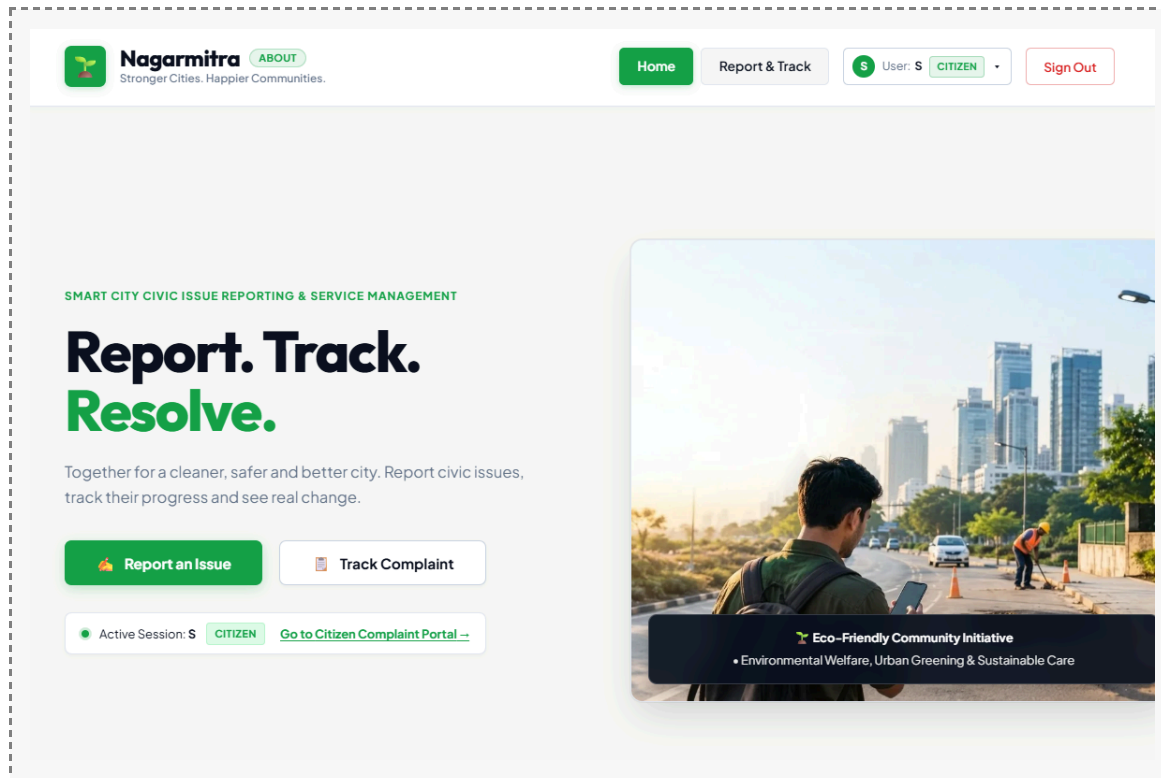


Figure 7.3: Citizen dashboard

7.3 Civic Issue Reporting

Citizens report a civic issue through the complaint form by entering a description and the type of issue, for example road damage, garbage, drainage, streetlight or water-related problem. The complaint is saved and appears in the citizen's complaint list.

Nagarmitra Stronger Cities. Happier Communities. [ABOUT](#)

Home **Report & Track** User: S **CITIZEN** [Sign Out](#)

[Back to Home](#) Municipal Citizen Services • Public Grievance Registration

File a Public Grievance

Submit photographic proof and street address. The AI engine evaluates urgency and dispatches the municipal department.

Issue Title *

e.g., Deep hazardous pothole near South Ave pedestrian crossing

Civic Category * **Responsible Department**

Pothole & Road Damage Roads & Infrastructure

Detailed Description & Hazards *

English (IN) हिन्दी (Hindi) తెలుగు (Telugu) Voice Dictate

Describe severity, approximate dimensions, water accumulation, traffic obstruction, or speak using the Voice Dictate button above...

Incident Location & Landmark *

Street name, landmark, ward or building number Pin on Map Auto-GPS

Attach Photographic Proof

Choose File No file chosen

Submit Civic Complaint

Municipal Reporting Standards

- Clear Photography:** Take wide-angle photos showing the defect alongside recognizable street landmarks.
- Precise Address:** Use the auto-GPS button to attach satellite coordinates for field workers.
- Duplicate Detection:** The AI vision model automatically links reports in the same 50m radius.
- Resolution SLA:** Critical road potholes and electrical hazards are assigned within 4 to 24 hours.

Already filed a complaint?

You have 3 tickets recorded in the municipal registry. Track live ground progress anytime.

[Track Active Issue](#) [View All History](#)

Figure 7.4: Complaint reporting form

7.4 Image Upload

While reporting a complaint, the citizen can upload images that show the problem. These images help the manager understand the issue before assigning it.

Incident Location & Landmark *

Street name, landmark, ward or building number Pin on Map Auto-GPS

Attach Photographic Proof

Choose File No file chosen

Figure 7.5: Image upload while reporting a complaint

7.5 Location-Based Reporting

The citizen marks the location of the issue on the map while submitting the complaint. The location is saved with the complaint and shown again when a manager or field worker opens it.

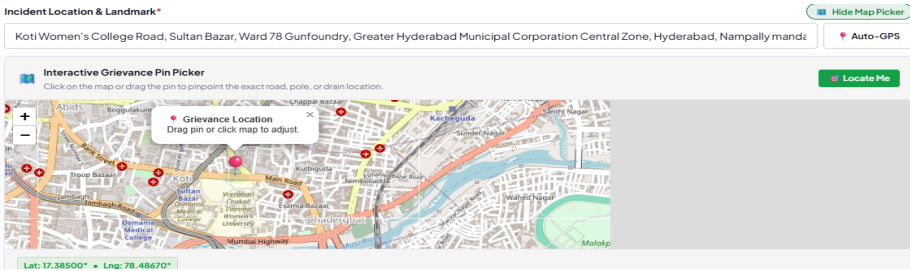


Figure 7.6: Location selection on the map

7.6 Complaint Tracking

Citizens can open their complaints and see the current status and progress. This gives them visibility of what is happening after they submit the complaint.

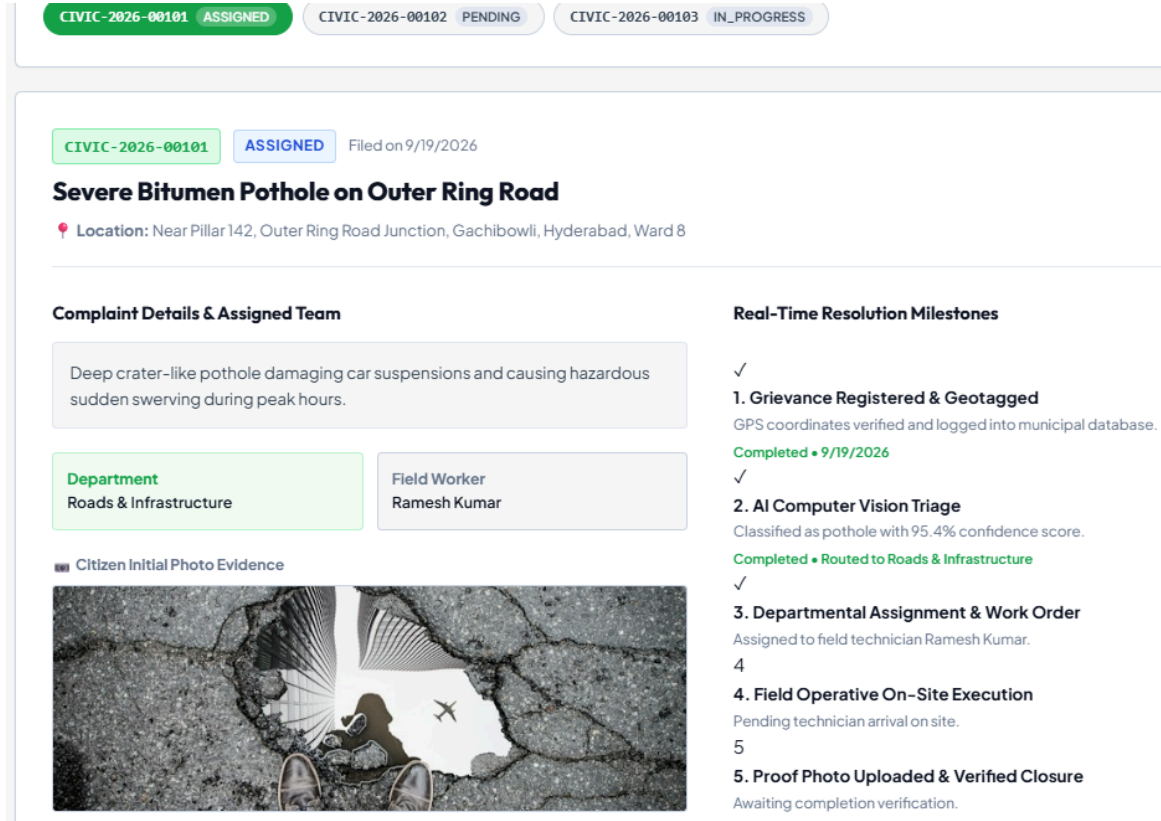


Figure 7.7: Complaint tracking page

7.7 Complaint Assignment

Managers view the complaints submitted by citizens, review the details, classify them and assign each one to a suitable department or field worker.

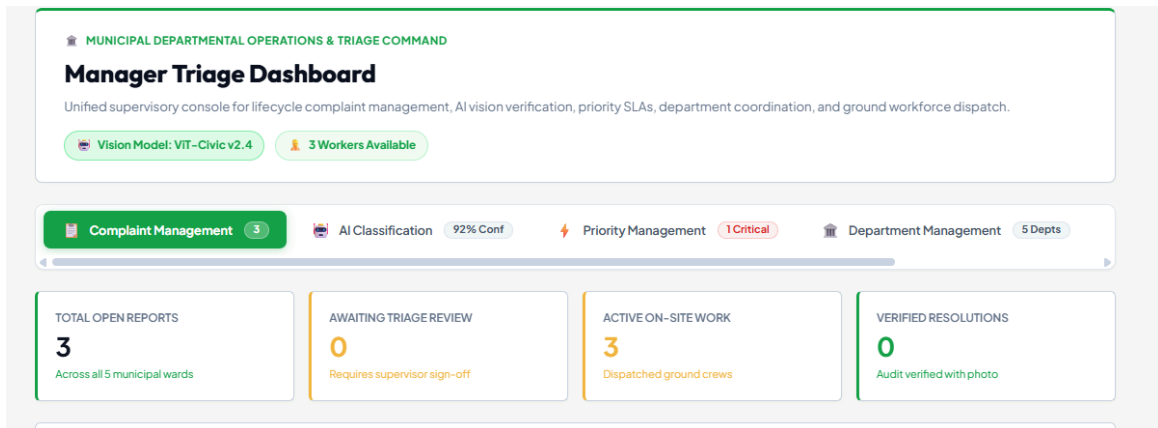


Figure 7.8: Manager dashboard with complaint assignment

7.8 Field Worker Updates

Field workers see the tasks assigned to them and update the status of a task as they work on it. The updated status becomes visible to the manager and the citizen.

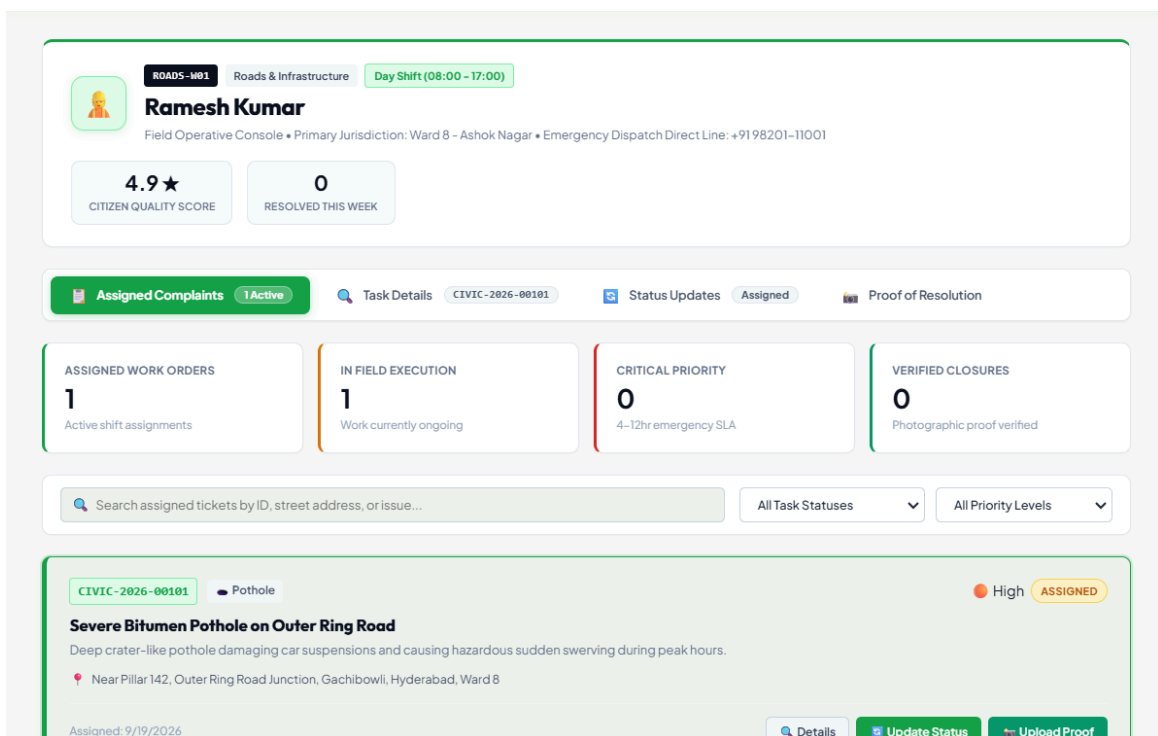


Figure 7.9: Field worker dashboard

7.9 Resolution Proof

After completing the work, the field worker submits proof of resolution, such as an image. This proof is stored with the complaint, so a complaint is not closed without evidence of the work.

MANDATORY PHOTOGRAPHIC RESOLUTION VERIFICATION • CIVIC-2026-00101

Resolution Proof: Severe Bitumen Pothole on Outer Ring Road

Municipal protocol requires verifiable before/after photographic records and materials accounting before complaint closure.

Before & After Visual Verification

DUAL PHOTOGRAPHIC RECORD

BEFORE: CITIZEN INCIDENT PHOTO	AFTER: FIELD OPERATIVE RESOLUTION PROOF
	 Attach Completed Work Photograph PNG, JPG, WebP up to 10MB Browse Photo Use Demo Photo
Date: 9/19/2026 Severity: HIGH	Operative: Ramesh Kumar Pending Upload

Figure 7.10: Resolution proof submission by field worker

7.10 Notifications

Users are informed about important events, such as the assignment of a complaint or a change in its status, through the notification panel of the application.

Citizen Alerts & Notifications
 You have 2 unread municipal updates.
 All 4
Unread 2
✓ Mark All as Read


Field Operative Dispatched NEW
1 hour ago

Ramesh Kumar from Roads & Infrastructure has been dispatched to repair your reported road defect.

[CIVIC-2026-00101](#) [Click to track live progress →](#)


AI Vision Triage Confirmed NEW
3 hours ago

Sanitation & Waste Management confirmed priority triage with 92.1% confidence score.

[CIVIC-2026-00102](#) [Click to track live progress →](#)


On-Site Work In Progress
Yesterday

Electrical crew is currently on-site at Shivaji Market Road repairing the exposed street pole wiring.

[CIVIC-2026-00103](#) [Click to track live progress →](#)


Resolution Proof Uploaded
Yesterday

Field worker Ramesh Kumar uploaded mandatory photographic proof of completed asphalt restoration.

[CIVIC-2026-00101](#) [Click to track live progress →](#)

Figure 7.11: Notification panel

7.11 Feedback

After a complaint is resolved, the citizen can submit feedback about the resolution. The feedback is stored against that complaint.



Citizen Satisfaction Feedback

Rate the quality, speed, and professionalism of municipal ground repairs.

Select Grievance to Review

CIVIC-2026-00101 - Severe Bitumen Pothole on Outer Ring Road (IN_PRO ▼)

Overall Experience Rating (1 to 5 Stars)



5 / 5 Stars

Resolution Speed

★★★★★

Repair Quality

★★★★★

Staff Behavior

★★★★★

Citizen Comments & Suggestions

Tell us about the completed work: was the site cleaned up? Did the issue resolve properly?



Submit Citizen Review

Figure 7.12: Citizen feedback form

7.12 Administrative Management

Administrators use the admin dashboard to manage users and view the overall data of the system.

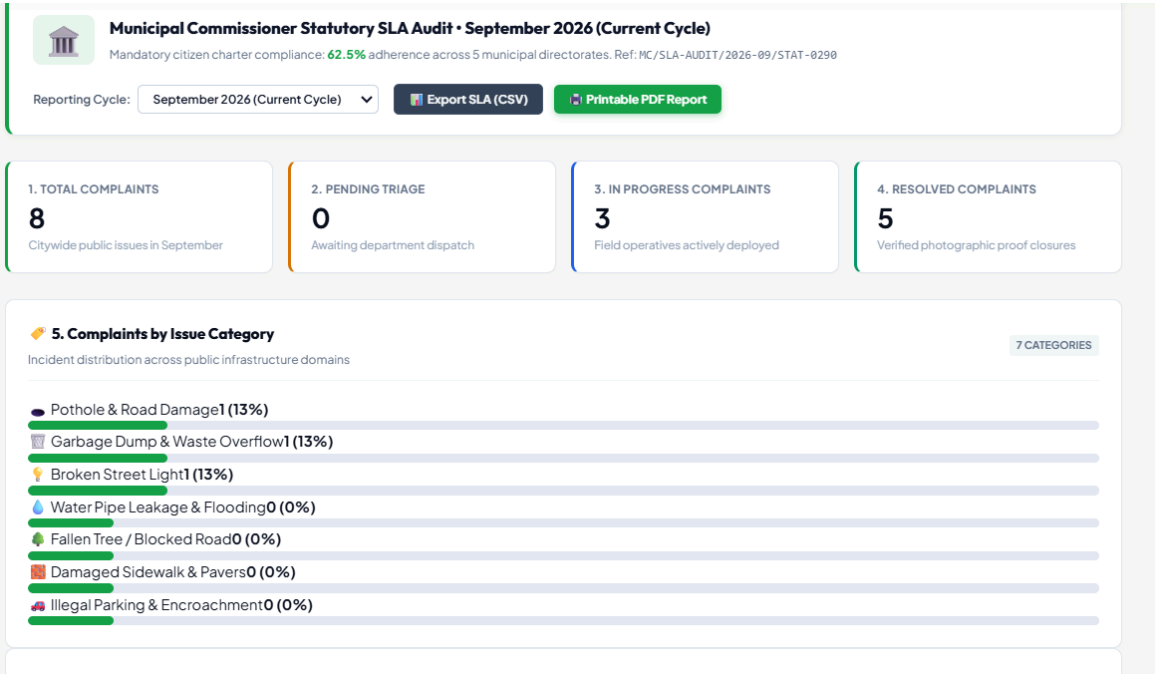


Figure 7.13: Admin dashboard

CHAPTER 8: TESTING

This chapter describes how Nagarmitra is tested and lists the test cases. The test cases and their expected results have been prepared from the features of the system. Wherever the actual result has not been recorded yet, it is marked as “TO BE UPDATED AFTER FINAL TESTING”. No test is reported as passed until its actual result is entered.

8.1 Frontend Testing

Frontend testing checks that the pages of the React application work as expected when used through the browser. It covers the following:

- Form validation on the registration, login, complaint and feedback pages.
- Correct display of data on the citizen, manager, field worker and admin dashboards.
- Working of buttons, links and navigation menus.
- Correct display of messages for success and error cases.

8.2 Responsive Testing

Responsive testing checks that the application layout remains usable on different screen sizes. The pages are opened on desktop, tablet and mobile widths, using the browser’s developer tools or actual devices.

Table 8.1: Responsive test screen categories

Device Category	Purpose of Check	Result
Desktop	Full layout of all dashboards and forms	Done
Tablet	Layout, menus and tables at medium width	Done
Mobile	Forms, navigation and readability at small width	Done

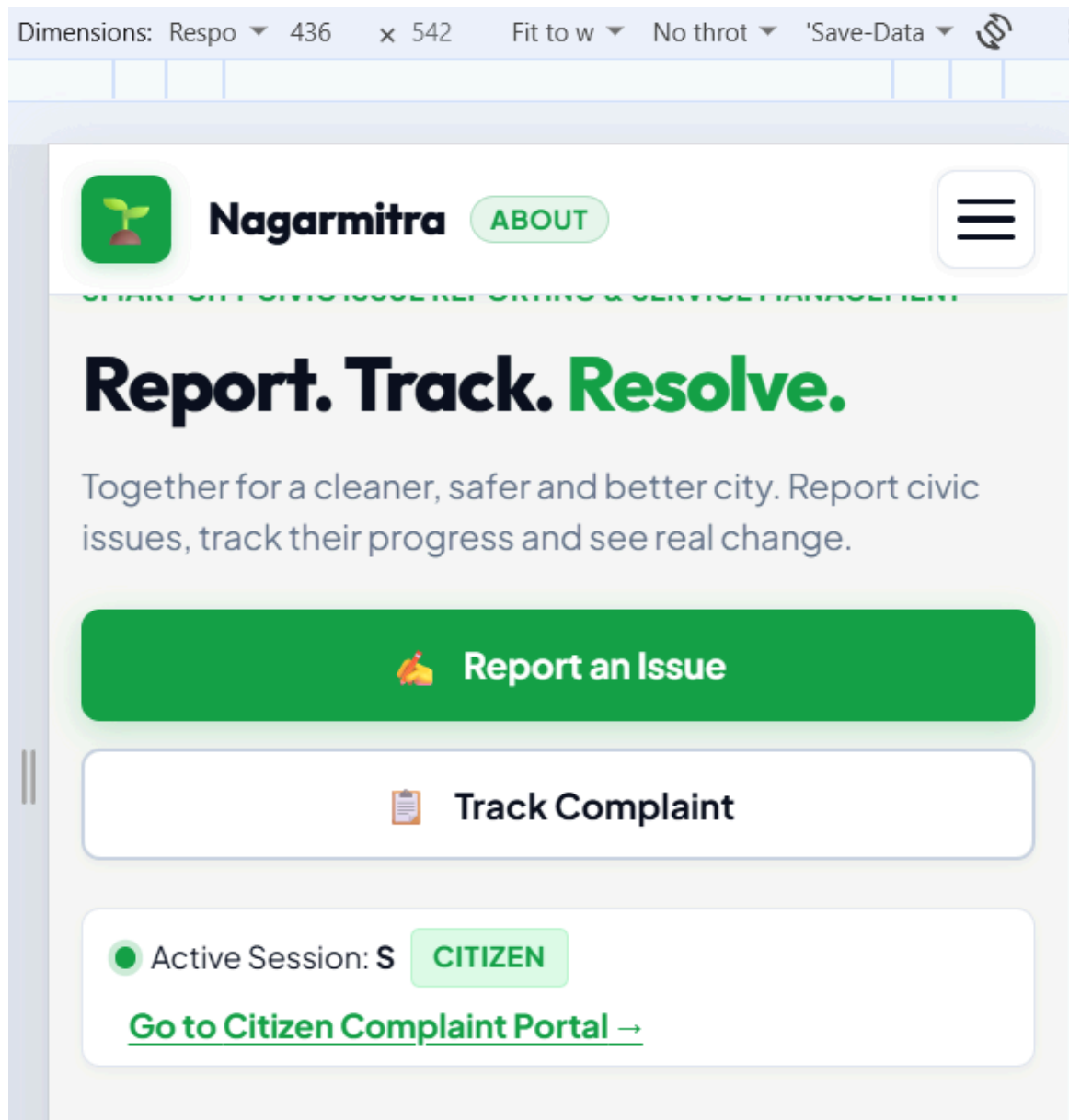


Figure 8.1: Mobile responsive view

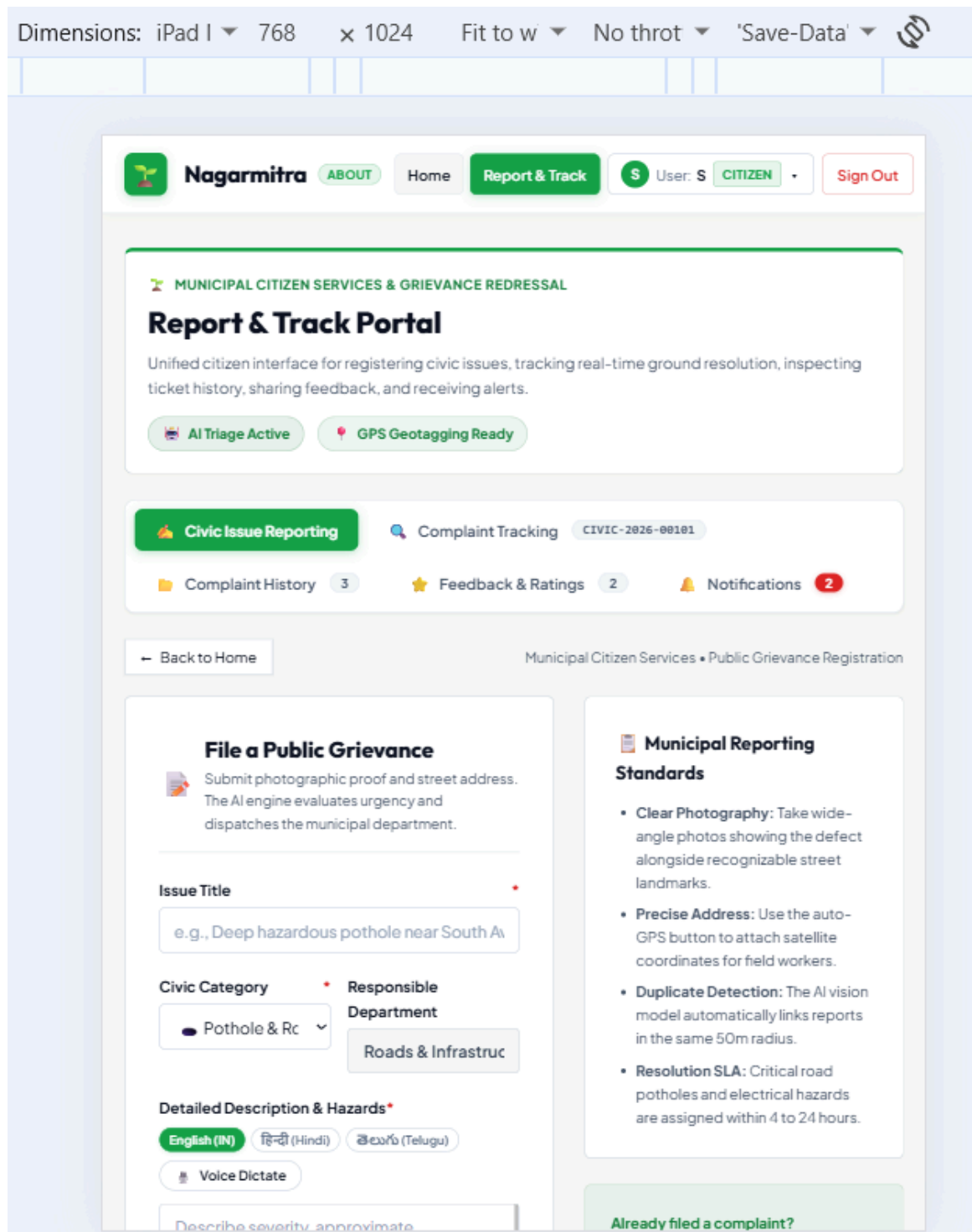


Figure 8.2: Tablet responsive view

8.3 API Testing Using Postman

Postman is used to send requests to the FastAPI endpoints without using the frontend. For each API, the request is sent with valid and invalid data, and the status code and response body are compared with the expected behaviour. Protected endpoints are also tested with and without a valid login to check access control.

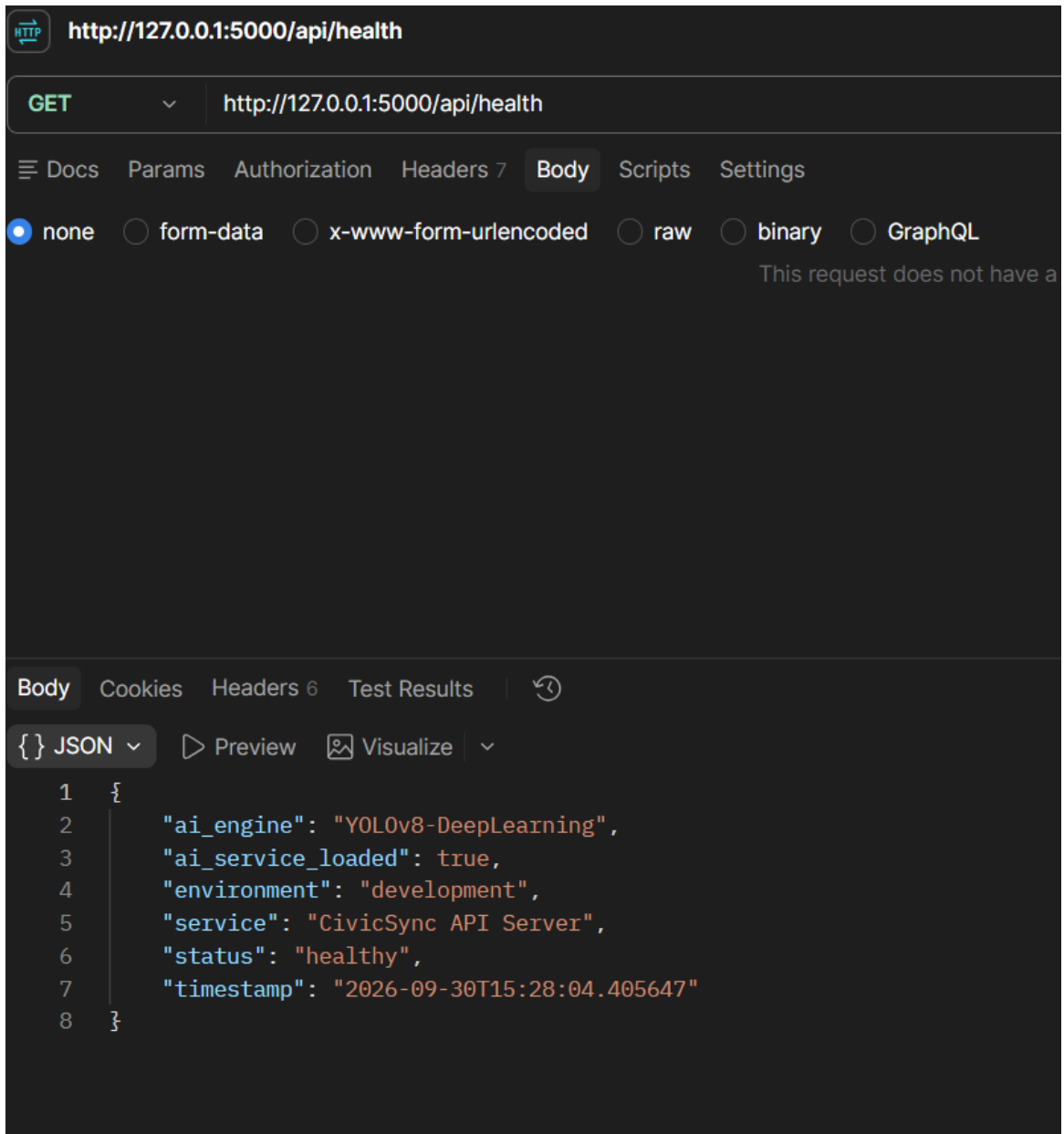


Figure 8.3: Sample API test in Postman

8.4 Postman/API Documentation Testing

FastAPI generates interactive documentation based on the OpenAPI standard. This documentation page (Swagger UI) is used to view the available endpoints, their request formats and responses, and to try them directly from the browser. It is also used to check that the documented endpoints match the actual behaviour of the backend.

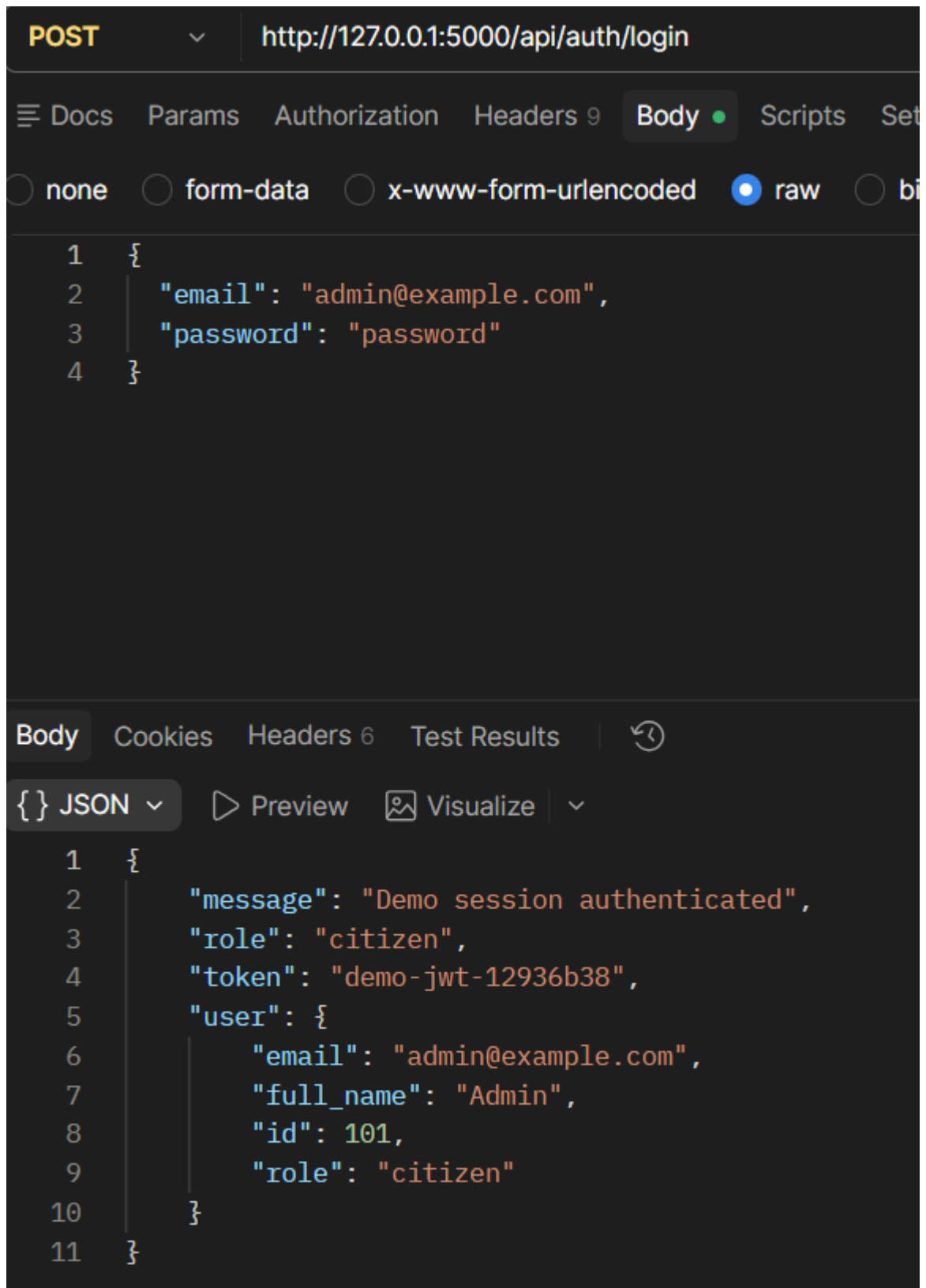
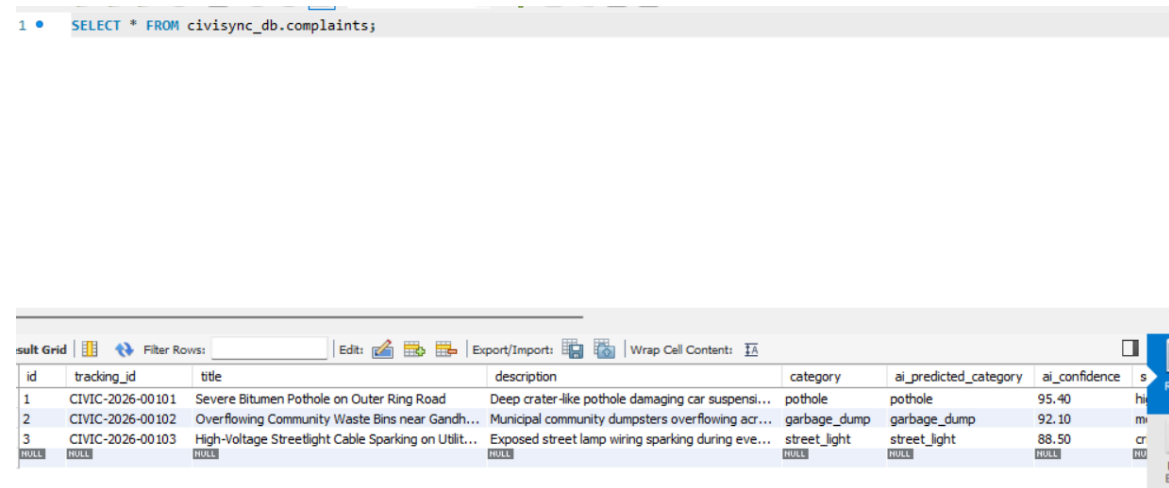


Figure 8.4: Postman Testing

8.5 Database Testing

Database testing checks that the data entered through the application is stored correctly in MySQL. After actions such as registering a user, submitting a complaint, assigning a task, updating status and saving feedback, the related tables are checked to confirm that the records are created or updated as expected and that the relations between tables are maintained.



The screenshot shows a MySQL database interface. At the top, a SQL query is entered: `SELECT * FROM civisync_db.complaints;`. Below the query, a table grid displays the results of the query. The table has columns: `id`, `tracking_id`, `title`, `description`, `category`, `ai_predicted_category`, `ai_confidence`, and `s`. The table contains three rows of data.

id	tracking_id	title	description	category	ai_predicted_category	ai_confidence	s
1	CIVIC-2026-00101	Severe Bitumen Pothole on Outer Ring Road	Deep crater-like pothole damaging car suspensi...	pothole	pothole	95.40	hi
2	CIVIC-2026-00102	Overflowing Community Waste Bins near Gandh...	Municipal community dumpsters overflowing acr...	garbage_dump	garbage_dump	92.10	m
3	CIVIC-2026-00103	High-Voltage Streetlight Cable Sparking on Utilit...	Exposed street lamp wiring sparking during eve...	street_light	street_light	88.50	cr

Figure 8.5: MySQL database tables and records

8.6 Frontend–Backend Integration Testing

Integration testing checks that the React frontend and the FastAPI backend work together. Complete flows are tested from the user interface, for example: a citizen submits a complaint, the manager assigns it, the field worker updates it and submits proof, and the citizen sees the updated status and gives feedback.

8.7 Authentication and Authorization Testing

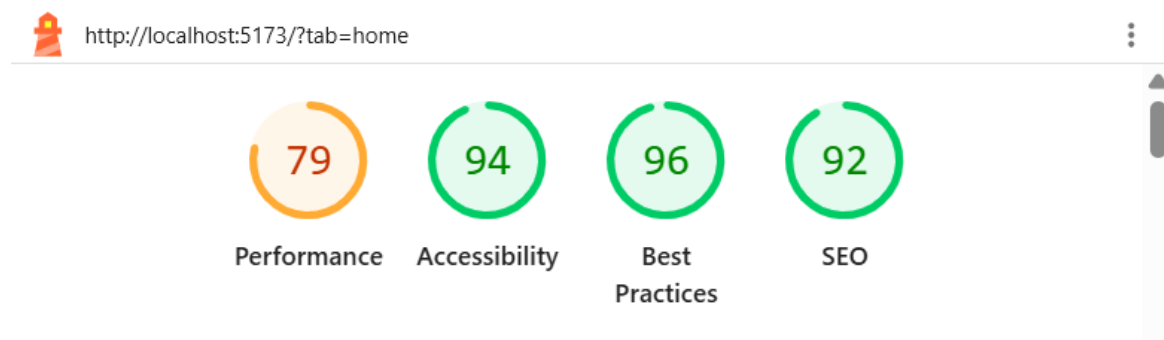
This testing checks that only valid users can log in and that each role can only use the functions meant for it. It includes login with correct and wrong details, access to role-specific pages by users of other roles, and access to protected APIs without a valid login.

8.8 Lighthouse / Performance Testing

Lighthouse, available in Chrome DevTools, is used to check the frontend for performance, accessibility, best practices and SEO. The report is generated for the main pages of the application and the scores are recorded after the final testing.

Table 8.2: Lighthouse report summary

Category	Score
Performance	80
Accessibility	94
Best Practices	96
SEO	92

**Figure 8.6: Lighthouse report**

CHAPTER 9: DEPLOYMENT AND SETUP

This chapter explains how Nagarmitra is set up and run on a local system. The project is run in a local development environment, and no hosting platform is described here unless it has actually been used.

9.1 Development Environment

The project was developed in Google Antigravity IDE. The frontend and backend are kept as separate parts of the project, and the source code is managed using Git and GitHub. The frontend needs Node.js and npm, the backend needs Python, and the database needs a MySQL server.

9.2 Frontend Setup

1. Install Node.js and npm.
2. Open a terminal in the frontend folder of the project.
3. Install the dependencies.
4. Start the React development server.

```
cd frontend
npm install
npm start # [CONFIRM: or npm run dev]
```

9.3 Backend Setup

1. Install Python 3.
2. Open a terminal in the backend folder of the project.
3. Create and activate a virtual environment (recommended).
4. Install the required Python packages.
5. Start the FastAPI server.

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt # [CONFIRM: file name]
uvicorn main:app --reload # [CONFIRM: module name]
```

9.4 MySQL Database Setup

1. Install MySQL Server and start the service.

2. Create a database for Nagarmitra.
3. Create the tables, or run the provided script or migration if one is available.
4. Enter the database details in the environment file of the backend.

```
CREATE DATABASE nagarmitra; -- [CONFIRM: database name]
```

9.5 Environment Variables

Values that differ between systems or must be kept private, such as database credentials, are placed in an environment file instead of being written in the code. The variables below are examples and should be replaced by the ones used in the project.

Table 9.1: Environment variables (to be confirmed)

Variable	Purpose
DB_HOST	Address of the MySQL server
DB_USER	MySQL user name
DB_PASSWORD	MySQL password
DB_NAME	Name of the database
[CONFIRM: other variables, e.g. secret key, map API key]	Other configuration values used by the project

9.6 Running the Complete Application

1. Start the MySQL server.
2. Start the FastAPI backend.
3. Start the React frontend.
4. Open the frontend address in a web browser.
5. Log in with an account of the required role.

9.7 API Server

When the FastAPI server is running, the interactive API documentation (Swagger UI) is available at the /docs path of the backend address. By default, FastAPI serves it at `http://127.0.0.1:8000/docs` when run locally on port 8000.

9.8 Frontend–Backend Connection

The frontend is configured with the address of the backend so that its API requests reach the FastAPI server. The backend must also allow requests from the frontend address (CORS) when they run on different ports.

CHAPTER 10: CHALLENGES AND LIMITATIONS

10.1 Challenges Faced During Development

Nagarmitra involves several roles, a frontend, a backend and a database, so the main challenge was keeping all parts consistent while they were being developed together. Decisions such as the complaint statuses, the data stored for each complaint and the actions allowed for each role had to be agreed on early, because changing them later affected the frontend, backend and database at the same time.

10.2 Frontend–Backend Integration Challenges

Connecting the React frontend to the FastAPI backend required the request and response formats to match exactly. Problems such as wrong field names, missing data and cross-origin (CORS) errors had to be identified and corrected. Handling file uploads together with normal form data also needed extra care.

10.3 Database Integration Challenges

Designing tables for users, complaints, assignments, status updates, notifications and feedback with correct relations took several revisions. Changes to the tables also needed matching changes in the backend code. Keeping test data consistent was another difficulty.

10.4 Role-Based Access Challenges

Each of the four roles has different pages and permissions. It was necessary to check the role in the backend for every protected API and not only hide the buttons in the frontend. Making sure that a user of one role could not reach the pages or data of another role required repeated checking.

10.5 Responsive Design Challenges

Dashboards with tables, forms and maps are difficult to fit on small screens. The layout had to be adjusted so that the pages remain usable on tablet and mobile widths.

10.6 API Testing Challenges

Testing APIs that depend on a logged-in user and a specific role needed several requests to be prepared in the correct order in Postman. Testing endpoints that accept images was also more involved than testing simple data requests.

10.7 Current Limitations

- Nagarmitra is a web application only; there is no separate mobile application.
- Complaints are classified by managers manually; there is no automatic classification.
- There is no automatic detection of duplicate complaints.
- Notifications are shown inside the application and are not real-time push notifications.
- Advanced analytics and reports are not part of the current version.
- The application is run in a local development environment and is not deployed on a cloud platform. It has not been integrated with other municipal or smart city systems.

CHAPTER 11: FUTURE ENHANCEMENTS

The items in this chapter are possible improvements for the future. They are not implemented in the current version of Nagarmitra.

11.1 Mobile Application

A mobile application could make it easier for citizens to report issues on the spot using the phone camera and location, and for field workers to update tasks while working outside.

11.2 Advanced AI-Based Complaint Classification

Complaint images and descriptions could be analysed automatically to suggest the issue type and the department. At present, this classification is done by managers.

11.3 Duplicate Complaint Detection

The system could detect when several citizens report the same issue at the same place and link these complaints together, so that the same work is not assigned more than once.

11.4 Real-Time Notifications

Notifications could be delivered instantly to the user, for example as push notifications, email or SMS, instead of only being visible inside the application.

11.5 Advanced Analytics

Dashboards and reports could show complaint trends, frequently affected areas, pending complaints and department performance to help administrators plan better.

11.6 Cloud Deployment

The application could be deployed on a cloud platform so that it can be accessed publicly and can handle more users.

11.7 Smart City Service Integration

Nagarmitra could be connected to other municipal or smart city services and official systems for wider service management.

11.8 Improved Location-Based Services

Location features could be extended, for example by showing nearby complaints on a map, grouping complaints by area, and assigning workers based on their distance from the issue.

CHAPTER 12: CONCLUSION

Nagarmitra addresses common problems in civic complaint handling, such as manual sorting of complaints, wrong routing, lack of visibility for citizens, poor connection with field workers and closure of complaints without proof of work. It brings the complete process into one web-based platform, from the time a citizen reports an issue to the time the citizen gives feedback on the result.

The system is built with React.js for the frontend, Python FastAPI for the backend and MySQL for the database. Maps and location features and image upload are used to describe the issue clearly, and the APIs are tested with Postman and Swagger/OpenAPI documentation.

The main users of the system are citizens, managers, field workers and administrators. The main workflow is that a citizen reports a complaint, a manager reviews and assigns it, a field worker updates the status and submits resolution proof, and the citizen tracks the progress, receives notifications and gives feedback.

The contribution of the project is a working model of how a municipal complaint process can be organised in one system with clear roles and a structured database. It is a student project, and it does not replace a full municipal system, but it shows how the process can be made more transparent and better connected.

While developing Nagarmitra, we learned how to build a web application with a separate frontend and backend, design a relational database, implement role-based access, use maps and image upload in an application, and test APIs using Postman and Swagger. We also learned how to work as a team using Git and GitHub, and how much planning is needed when several parts of a system depend on each other. The features described as future enhancements can be added later to extend the system.

CHAPTER 13: REFERENCES

1. React Documentation. <https://react.dev>
2. Python Documentation. <https://docs.python.org/3/>
3. FastAPI Documentation. <https://fastapi.tiangolo.com>
4. MySQL Reference Manual. <https://dev.mysql.com/doc/>
5. Postman Documentation. <https://learning.postman.com/docs/>
6. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>
7. Swagger Documentation. <https://swagger.io/docs/>
8. MDN Web Docs. <https://developer.mozilla.org>
9. Lighthouse Documentation, Chrome for Developers.
<https://developer.chrome.com/docs/lighthouse/overview>
10. Git Documentation. <https://git-scm.com/doc>
11. GitHub Docs. <https://docs.github.com>

APPENDIX A: APPLICATION SCREENSHOTS

This appendix contains the main screens of Nagarmitra.

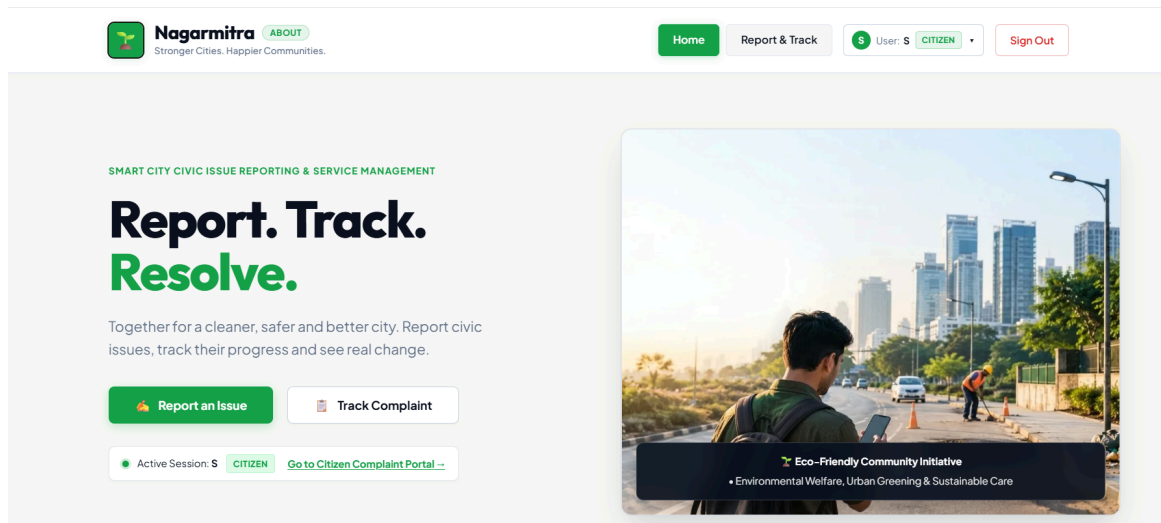


Figure A.1: Nagarmitra home page

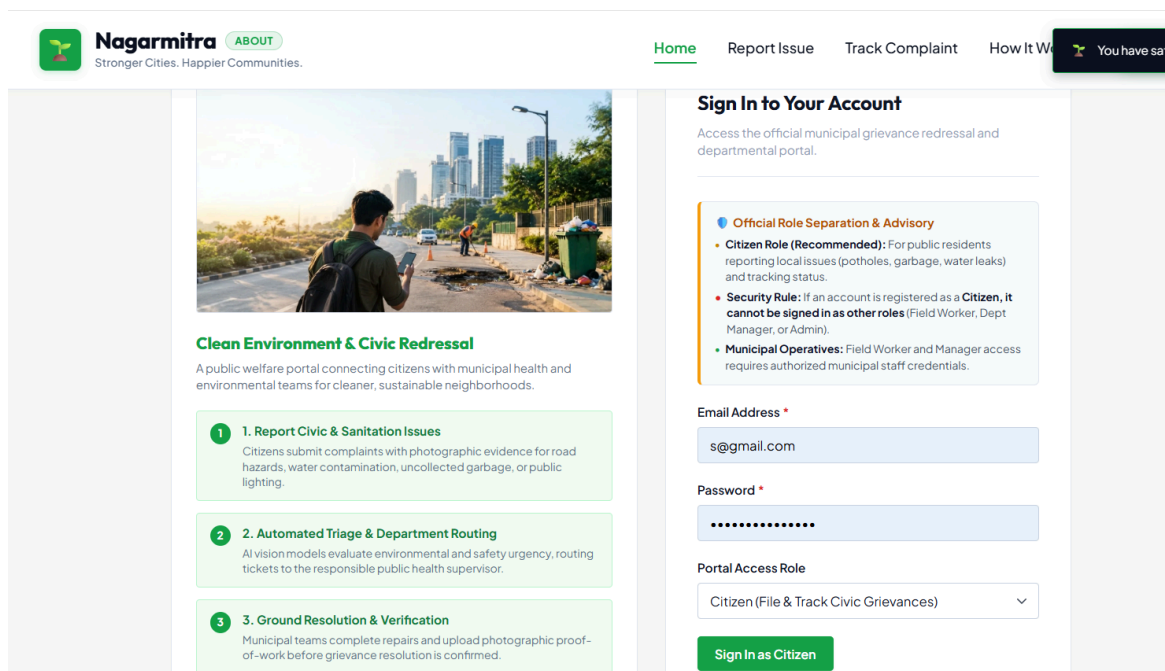


Figure A.2: Citizen login

Nagarmitra ABOUT
Stronger Cities. Happier Communities.

Home Report Issue Track Complaint How It Works You have saved 0 items

Clean Environment & Civic Redressal

A public welfare portal connecting citizens with municipal health and environmental teams for cleaner, sustainable neighborhoods.

- 1. Report Civic & Sanitation Issues**
Citizens submit complaints with photographic evidence for road hazards, water contamination, uncollected garbage, or public lighting.
- 2. Automated Triage & Department Routing**
AI vision models evaluate environmental and safety urgency, routing tickets to the responsible public health supervisor.
- 3. Ground Resolution & Verification**
Municipal teams complete repairs and upload photographic proof-of-work before grievance resolution is confirmed.

Sign In to Your Account

Access the official municipal grievance redressal and departmental portal.

Official Role Separation & Advisory

- Citizen Role (Recommended):** For public residents reporting local issues (potholes, garbage, water leaks) and tracking status.
- Security Rule:** If an account is registered as a **Citizen**, it **cannot be signed in as other roles** (Field Worker, Dept Manager, or Admin).
- Municipal Operatives:** Field Worker and Manager access requires authorized municipal staff credentials.

Email Address *
s@gmail.com

Password *
.....

Portal Access Role
Citizen (File & Track Civic Grievances) ▾

Sign In as Citizen

Figure A.3: Citizen dashboard

Civic Issue Reporting Complaint Tracking CIVIC-2026-00101 Complaint History 3 Feedback & Ratings 2 Notifications 2

Back to Home Municipal Citizen Services • Public Grievance Registration

File a Public Grievance

Submit photographic proof and street address. The AI engine evaluates urgency and dispatches the municipal department.

Issue Title
e.g., Deep hazardous pothole near South Ave pedestrian crossing

Please fill out this field.

Civic Category
Pothole & Road Damage

Responsible Department
Roads & Infrastructure

Detailed Description & Hazards * English (IN) हिन्दी (Hindi) తెలుగు (Telugu) Voice Dictate

Describe severity, approximate dimensions, water accumulation, traffic obstruction, or speak using the Voice Dictate button above...

Municipal Reporting Standards

- Clear Photography:** Take wide-angle photos showing the defect alongside recognizable street landmarks.
- Precise Address:** Use the auto-GPS button to attach satellite coordinates for field workers.
- Duplicate Detection:** The AI vision model automatically links reports in the same 50m radius.
- Resolution SLA:** Critical road potholes and electrical hazards are assigned within 4 to 24 hours.

Already filed a complaint?
You have 3 tickets recorded in the municipal registry. Track live ground progress anytime.

Track Active Issue **View All History**

Figure A.4: Complaint reporting form

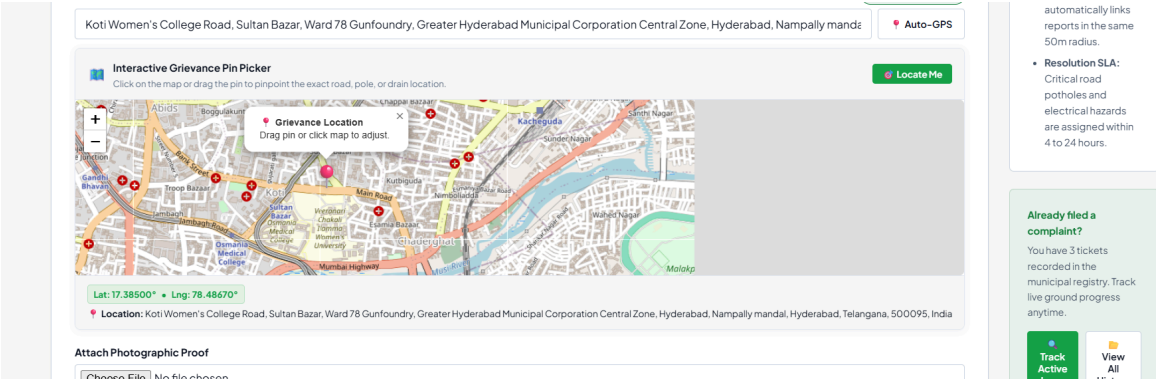


Figure A.5: Map location selection

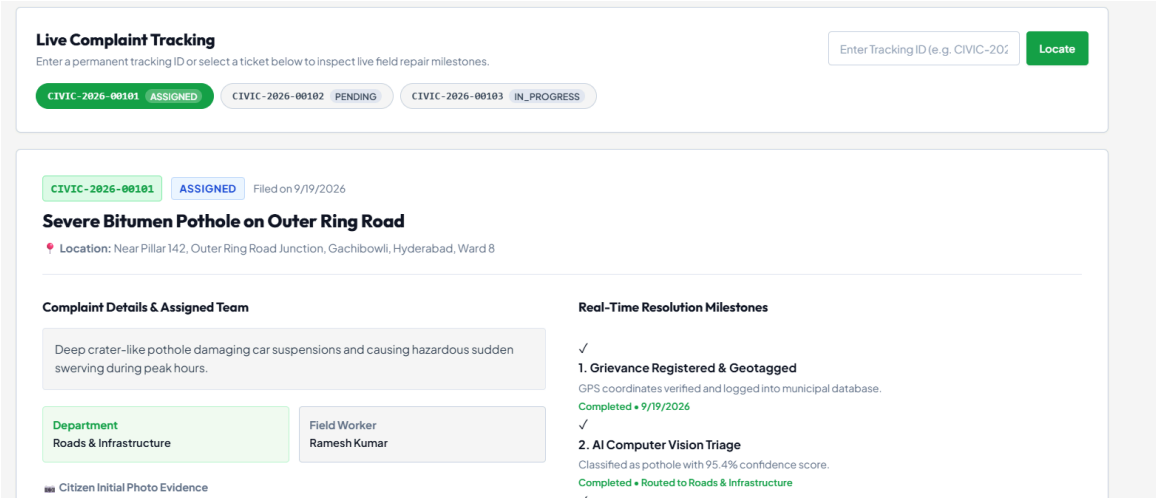


Figure A.6: Complaint tracking

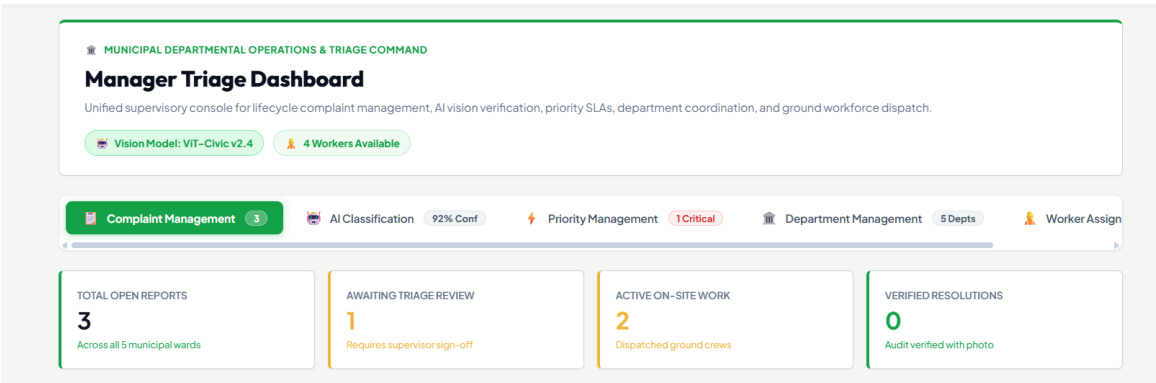


Figure A.7: Manager dashboard

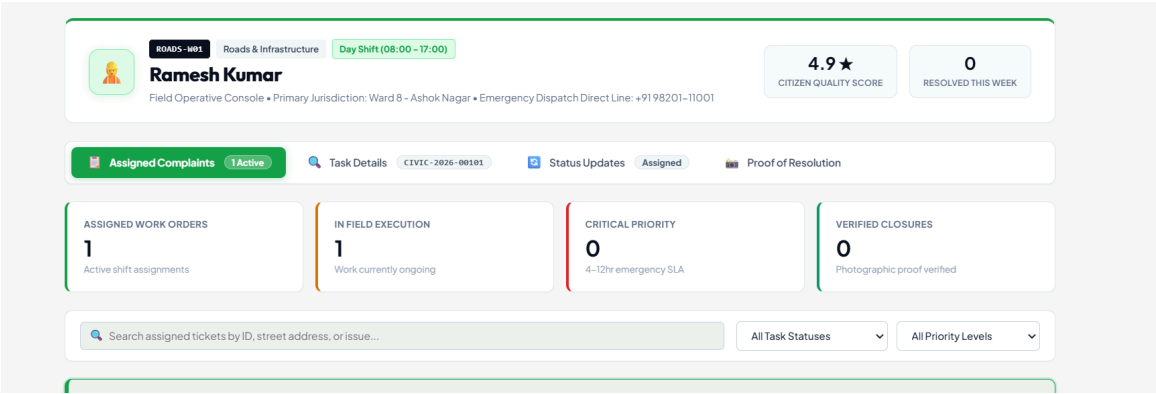


Figure A.8: Field worker dashboard

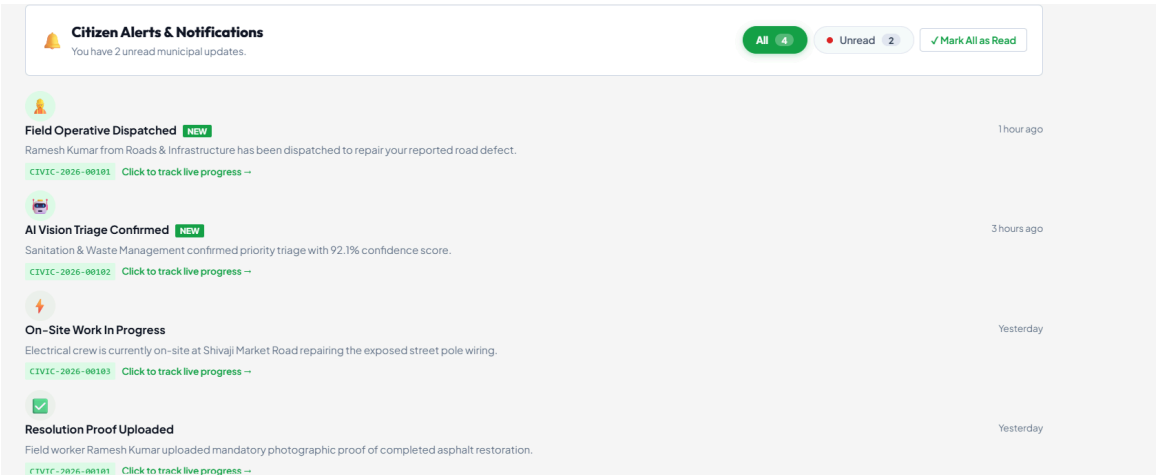


Figure A.9: Notification panel

APPENDIX B: API TESTING SCREENSHOTS

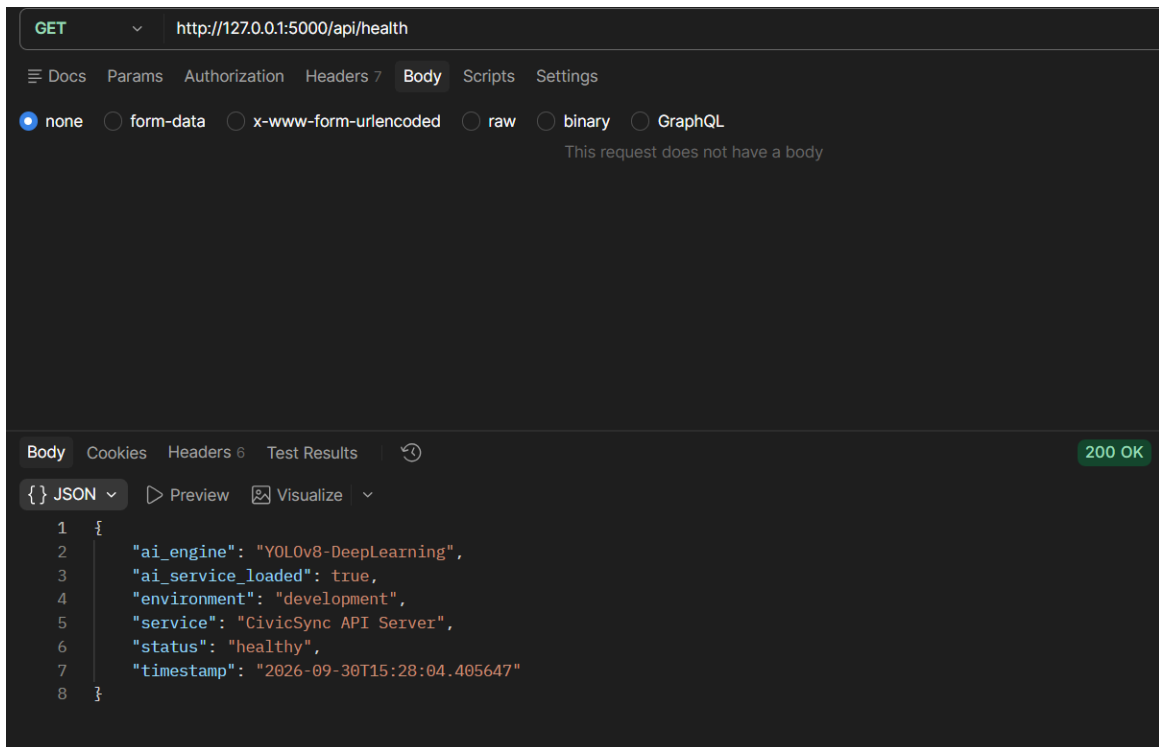


Figure B.1: Postman API test

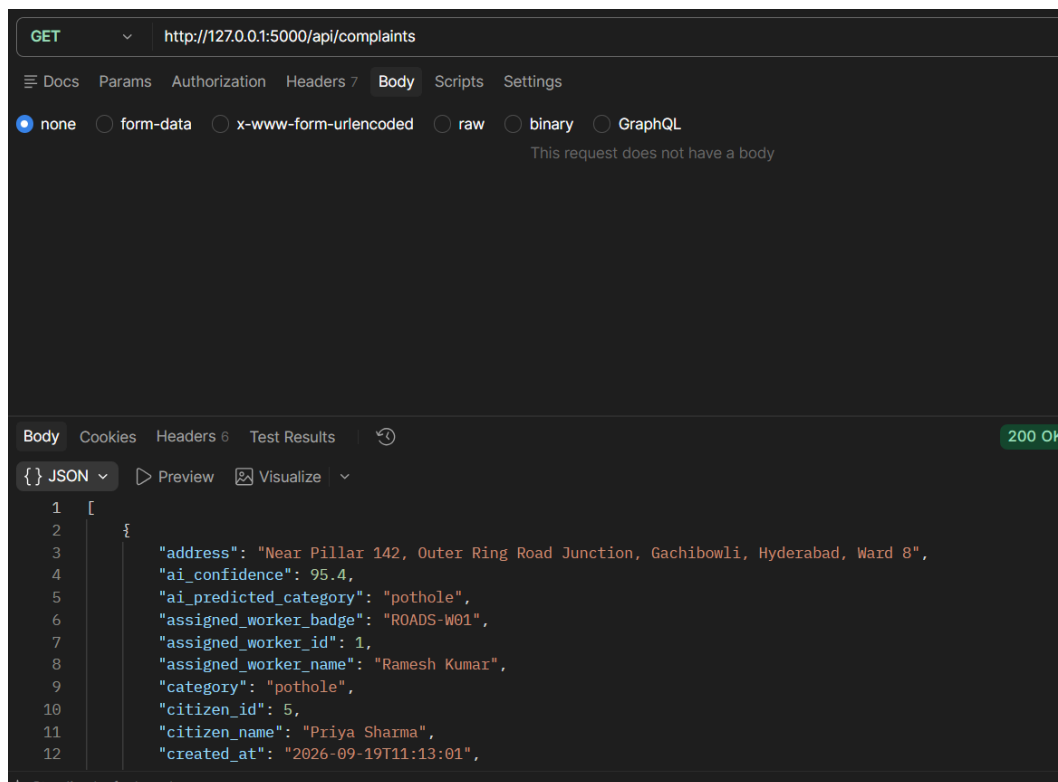
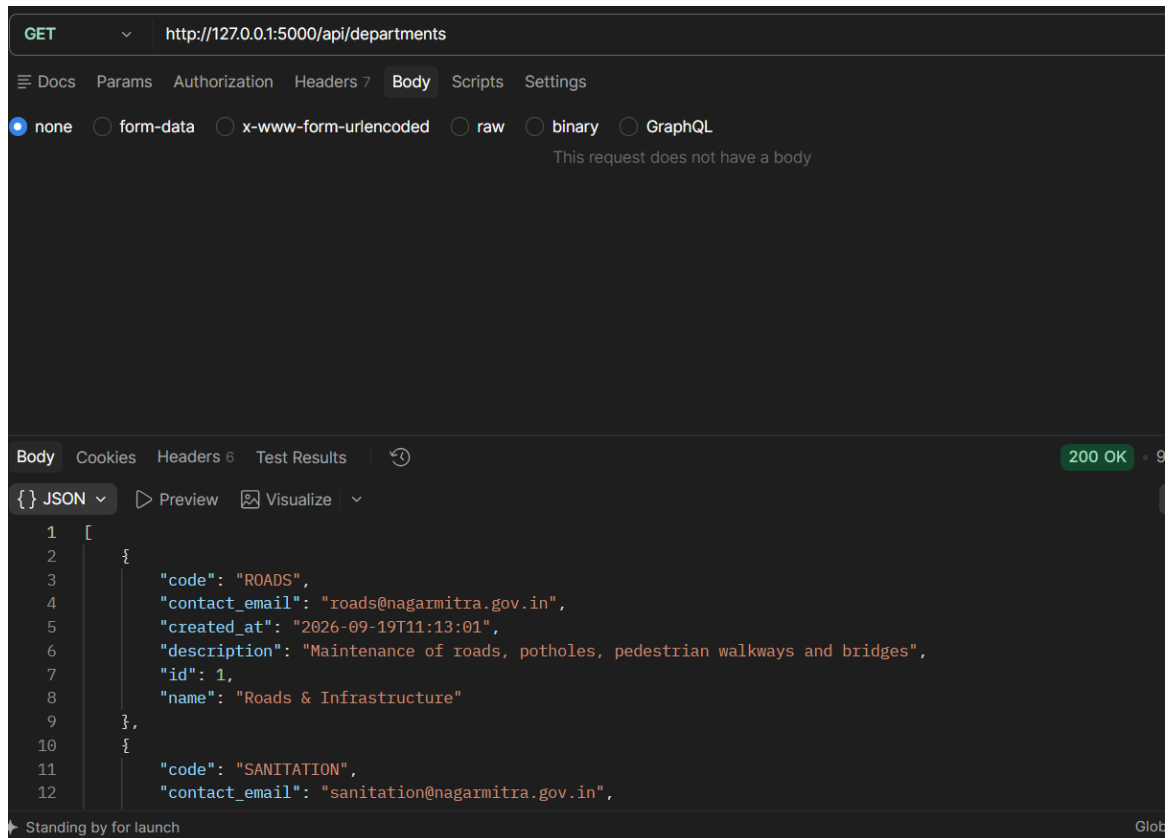


Figure B.2: Postman



APPENDIX C: DATABASE SCREENSHOTS

1 • `SELECT * FROM civisync_db.complaints;`

Result Grid

Filter Rows:

Edit

Export/Import

Wrap Cell Content

	id	tracking_id	title	description	category	ai_predicted_category	ai_confidence	s
▶	1	CIVIC-2026-00101	Severe Bitumen Pothole on Outer Ring Road	Deep crater-like pothole damaging car suspensi...	pothole	pothole	95.40	hi
	2	CIVIC-2026-00102	Overflowing Community Waste Bins near Gandh...	Municipal community dumpsters overflowing acr...	garbage_dump	garbage_dump	92.10	m
	3	CIVIC-2026-00103	High-Voltage Streetlight Cable Sparking on Utilt...	Exposed street lamp wiring sparking during eve...	street_light	street_light	88.50	cr
•		NULL	NULL	NULL	NULL	NULL	NULL	NO

complaints 1 x

Apply Revert

Figure C.1: MySQL database tables

APPENDIX D: RESPONSIVE TESTING SCREENSHOTS

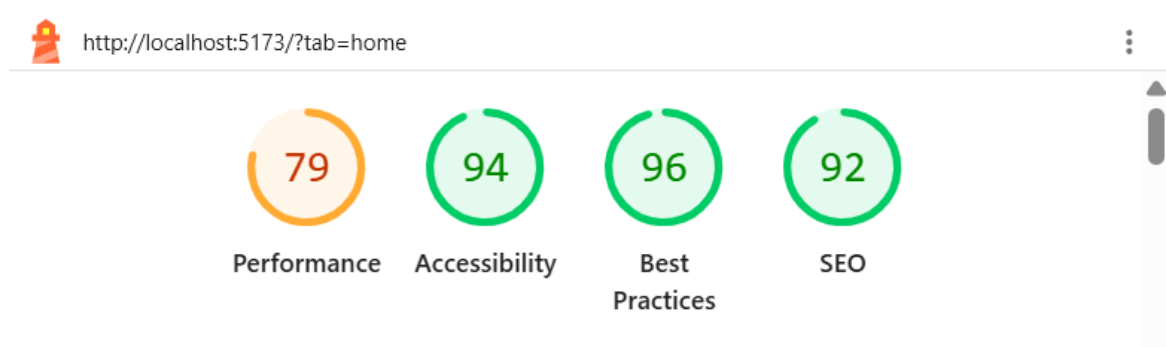


Figure D.3: Lighthouse report

Chapter 14 - Code Screenshots

```
export const OFFICIAL_LANGUAGES = [
];

/**
 * Nagarmitra Official Government/Municipal Portal Sign Up
 * First asks the user which language is suitable (Step 1),
 * then collects account credentials and role designation (Step 2).
 */
export default function SignUp({ onRegisterSuccess, onSwitchToSignIn }) {
  // Step 1: Language Preference, Step 2: Account Details
  const [step, setStep] = useState(1);
  const [selectedLang, setSelectedLang] = useState(OFFICIAL_LANGUAGES[0]);

  const [fullName, setFullName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [role, setRole] = useState('citizen');
  const [employeeCode, setEmployeeCode] = useState('');
  const [agreeTerms, setAgreeTerms] = useState(true);
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState(null);
  const [successMsg, setSuccessMsg] = useState(null);

  const handleContinueToStep2 = () => {
    setErrorMsg(null);
    setStep(2);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setErrorMsg(null);
```

Signup page

```
import React, { useState, useEffect } from 'react';
import CivicInfoPanel from './CivicInfoPanel.jsx';
import {
  checkRoleAllowedForEmail,
  recordUserRole,
  DEMO_CREDENTIALS,
  getRoleDisplayName,
} from '../utils/authRoles.js';

/**
 * Nagarmitra Official Government/Municipal Portal Sign In
 * Clean, institutional look with strict municipal role segregation.
 * Enforces rule: Users registered as Citizen cannot sign in as Field Worker or other official roles.
 */
export default function SignIn({ onLoginSuccess, onSwitchToSignUp }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [selectedRole, setSelectedRole] = useState('citizen');
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState(null);
  const [successMsg, setSuccessMsg] = useState(null);
  const [roleWarning, setRoleWarning] = useState(null);

  // Real-time check when user types email or changes role
  useEffect(() => {
    if (email.trim() && email.includes('@')) {
      const check = checkRoleAllowedForEmail(email, selectedRole);
      if (!check.allowed) {
        setRoleWarning(check.message);
      } else {
        setRoleWarning(null);
      }
    }
  });
}
```

Sign in page

```
export default function HomePage({ currentUser, onSwitchToSignUp }) {
  // Key municipal infrastructure & public health domains with real stock photography
  const DOMAINS = [
    {
      title: 'Roads & Public Pavements',
      dept: 'Roads & Infrastructure Department',
      desc: 'Repairing potholes, damaged asphalt, sinking manholes, and cracked pedestrian footpaths across urban wards.',
      imgUrl: 'https://images.unsplash.com/photo-1590486803833-1c5dc8ddd4c8?w=800',
      caption: 'Municipal asphalt paving & road restoration crew at work.',
      actionTab: 'citizen',
    },
    {
      title: 'Sanitation & Solid Waste',
      dept: 'Sanitation & Environmental Welfare',
      desc: 'Clearing overflowing public dumpsters, illegal trash heaps, and coordinating daily neighborhood waste removal.',
      imgUrl: 'https://images.unsplash.com/photo-1605600659908-0ef719419d41?w=800',
      caption: 'Urban municipal solid waste collection and street cleaning.',
      actionTab: 'citizen',
    },
    {
      title: 'Street Lighting & Energy',
      dept: 'Electrical & Energy Conservation',
      desc: 'Rectifying non-functional streetlights, damaged lamp posts, hanging electrical cables, and dark public corridors.',
      imgUrl: 'https://images.unsplash.com/photo-1509198397868-475647b2a1e5?w=800',
      caption: 'Public lighting utility maintenance and grid inspection.',
      actionTab: 'citizen',
    },
    {
      title: 'Water Supply & Green Corridors',
      dept: 'Water Supply & Urban Drainage',
      desc: 'Fixing burst water mainlines, flooded street intersections, clogged storm drains, and broken curb pavers.',
      imgUrl: 'https://images.unsplash.com/photo-1574958269340-fa927503f3dd?w=800',
      caption: 'Clean, accessible municipal sidewalks and urban storm drainage.',
      actionTab: 'citizen',
    },
  ];

  // Helper to determine role specific landing portal
  const getRoleDestination = () => {

```

```

import React, { useState, useEffect, useMemo } from 'react';
import CitizenNavBar from './citizen/CitizenNavBar.jsx';
import CivicIssueReporting from './citizen/CivicIssueReporting.jsx';
import ComplaintTracking from './citizen/ComplaintTracking.jsx';
import ComplaintHistory from './citizen/ComplaintHistory.jsx';
import CitizenFeedback from './citizen/CitizenFeedback.jsx';
import CitizenNotifications from './citizen/CitizenNotifications.jsx';

/**
 * Nagarmitra Citizen Service Portal (Report & Track)
 * -----
 * Modular Coordinator Component unifying 5 civic modules:
 * 1. Civic Issue Reporting (Form with AI Vision Pre-scan & GPS)
 * 2. Complaint Tracking (Live 5-stage milestone resolution tracker & photo verification)
 * 3. Complaint History (Searchable historical ticket log with status filters)
 * 4. Citizen Feedback (Post-resolution rating system with municipal review acknowledgment)
 * 5. Notifications (Alerts inbox with unread counter and direct ticket navigation)
 */
export default function CitizenDashboard({
  complaints = [],
  onAddComplaint,
  onNotification,
  initialSubTab = 'reporting',
  onSubTabChange,
  onSwitchTab,
  currentUser,
}) {
  // Active module tab: 'reporting' | 'tracking' | 'history' | 'feedback' | 'notifications'
  const [activeTab, setActiveTab] = useState(initialSubTab);
  const [recentlyCreatedTicket, setRecentlyCreatedTicket] = useState(null);

  useEffect(() => {
    if (initialSubTab && initialSubTab !== activeTab) {
      setActiveTab(initialSubTab);
    }
  }, [initialSubTab]);

  const handleTabSwitch = (tab) => {
    setActiveTab(tab);
    if (onSwitchTab) {
      onSwitchTab('citizen', tab);
    }
  };

```

Citizen Dashboard

```

import React, { useState, useMemo } from 'react';
import { calculateSLAMetrics, downloadSLACSV } from '../utils/slaReportGenerator.js';
import SLAReportModal from './admin/SLAReportModal.jsx';

/**
 * Nagarmitra Municipal Executive & Administrative Dashboard
 * Displays cross-department metrics, SLA compliance, worker dispatch readiness,
 * and quick-access links to citizen and manager triage views.
 */
export default function AdminDashboard({ complaints = [], onSwitchTab, onNotification }) {
  const [selectedPeriodKey, setSelectedPeriodKey] = useState('2026-09');
  const [isPrintModalOpen, setIsPrintModalOpen] = useState(false);

  const slaData = useMemo(() => {
    return calculateSLAMetrics(complaints, selectedPeriodKey);
  }, [complaints, selectedPeriodKey]);

  const totalTickets = slaData.executive.totalVolume;
  const pendingTickets = slaData.executive.totalPending;
  const inProgressTickets = slaData.executive.totalInProgress;
  const resolvedTickets = slaData.executive.totalResolved;
  const resolutionRate = slaData.executive.resolutionRate;

  const departments = slaData.departments.map((dept) => {
    const budgets = {
      roads: '₹4.8 Cr',
      sanitation: '₹6.2 Cr',
      electrical: '₹2.9 Cr',
      water: '₹5.5 Cr',
      parks: '₹1.8 Cr',
    };
    return {
      name: dept.name,
      lead: dept.lead,
      open: dept.totalVolume - dept.resolvedCount,
      slaRate: `${dept.complianceRate}%`,
      budget: budgets[dept.id] || '₹2.5 Cr',
      rating: dept.rating,
    };
  });
}

```

Admin page

```

export default function ManagerDashboard({
  onUpdateComplaint,
  onNotification,
  initialModule = 'complaints',
  onModuleChange,
}) {
  // Active module: 'complaints' | 'ai' | 'priority' | 'departments' | 'workers'
  const [activeTab, setActiveTab] = useState(initialModule);

  useEffect(() => {
    if (initialModule) {
      setActiveTab(initialModule);
    }
  }, [initialModule]);

  const handleTabSwitch = (tab) => {
    setActiveTab(tab);
    if (onModuleChange) onModuleChange(tab);
  };

  // Selected ticket for detailed review
  const [selectedTicketId, setSelectedTicketId] = useState(() => {
    return complaints.length > 0 ? complaints[0].id : null;
  });

  const selectedTicket = useMemo(() => {
    return complaints.find((c) => c.id === selectedTicketId) || complaints[0] || null;
  }, [complaints, selectedTicketId]);

  // Workforce Directory State
  const [workers, setWorkers] = useState([
    {
      id: 1,
      name: 'Ramesh Kumar',
      department: 'Roads & Infrastructure',
      badge: 'ROADS-W01',
      status: 'available',
      phone: '+91 98201-11001',
      ward: 'Ward 8 Ashok Nagar',
      activeTickets: 1,
    },
  ]);

```

Manager dashboard page

```
import React, { useState, useEffect, useMemo } from 'react';
import FieldOpsNavBar from './field-operations/FieldOpsNavBar.jsx';
import AssignedComplaints from './field-operations/AssignedComplaints.jsx';
import TaskDetails from './field-operations/TaskDetails.jsx';
import StatusUpdates from './field-operations/StatusUpdates.jsx';
import ProofOfResolution from './field-operations/ProofOfResolution.jsx';

/**
 * Field Operations Coordinator
 * Connects the 4 dedicated modular sub-components:
 * 1. Assigned Complaints (Queue & Filters)
 * 2. Task Details (Site Telemetry, Safety PPE & Citizen Evidence)
 * 3. Status Updates (Field Lifecycle Transitions & Activity Log)
 * 4. Proof of Resolution (Before/After Photo Verification & Signoff)
 */
export default function WorkerDashboard({
  complaints = [],
  onUpdateComplaint,
  onNotification,
  initialModule = 'assigned',
  onModuleChange,
}) {
  // Current logged in field operative profile
  const workerProfile = {
    name: 'Ramesh Kumar',
    badge: 'ROADS-W01',
    department: 'Roads & Infrastructure',
    phone: '+91 98201-11001',
    rating: '4.9 ★',
    activeShift: 'Day Shift (08:00 - 17:00)',
    ward: 'Ward 8 - Ashok Nagar',
  };

  // Active module tab: 'assigned' | 'details' | 'status' | 'proof'
  const [activeTab, setActiveTab] = useState(initialModule);

  useEffect(() => {
    if (initialModule) {
      setActiveTab(initialModule);
    }
  }, [initialModule]);
}
```

Worker dashboard page

```
import React from 'react';

/**
 * AboutPage Component
 * Official Municipal Public Service Overview & Citizen Charter for Nagarmitra
 * Accessible by all 4 roles and guests.
 */
export default function AboutPage({ onSwitchTab, currentUser }) {
  const steps = [
    {
      num: '01',
      icon: '📷',
      title: 'Geotagged Incident Reporting',
      desc: 'Citizens capture real-time photographic evidence with device GPS coordinates, flagging public infrastructure d
    },
    {
      num: '02',
      icon: '👁️',
      title: 'AI Computer Vision Triage',
      desc: 'ViT-Civic neural network analyzes images with 96.4% accuracy, categorizing defects (potholes, waste, lighting,
    },
    {
      num: '03',
      icon: '🚒',
      title: 'Departmental Dispatch',
      desc: 'Municipal department leads review statutory SLAs, allocate specialized fleet equipment, and dispatch trained f
    },
    {
      num: '04',
      icon: '👤',
      title: 'On-Site Field Resolution',
      desc: 'Field operatives deploy safety perimeters, execute repairs, and upload mandatory photographic proof of resolut
    },
    {
      num: '05',
      icon: '★',
      title: 'Citizen Verification & Closure',
      desc: 'Citizens inspect before-and-after proof photos, rate municipal repair quality, and confirm neighborhood restor
    },
  ],
}
```

About page

```

import React, { useState, useEffect } from 'react';
import { NAV_LINKS_BY_ROLE } from '../utils/roleConfig.js';
import { getTabUrl } from '../utils/navigation.js';

/**
 * Official Government / Municipal Navigation Header for Nagarmitra
 * Social Domain & Eco-Civic Theme (Clean Emerald & Sage Institutional Palette)
 * Features standard text links, green civic seal emblem, and clear unauthenticated/authenticated actions.
 */
export default function Navbar({
  currentUser,
  authView,
  onSwitchAuthView,
  onSignOut,
  currentTab,
  onSwitchTab,
  pendingCount = 0,
  managerModule = 'complaints',
  workerModule = 'assigned',
  citizenSubTab = 'reporting',
  backendHealth = 'checking', → Tab to Jump
}) {
  const [managerMenuOpen, setManagerMenuOpen] = useState(false);
  const [workerMenuOpen, setWorkerMenuOpen] = useState(false);
  const [userMenuOpen, setUserMenuOpen] = useState(false);
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);

  // Close dropdowns on outside click or Escape key
  useEffect(() => {
    const handleGlobalClick = (e) => {
      if (
        !e.target.closest('.govt-nav-dropdown-wrap') &&
        !e.target.closest('.govt-nav-user-wrap') &&
        !e.target.closest('.govt-hamburger-btn') &&
        !e.target.closest('.govt-mobile-drawer')
      ) {
        setManagerMenuOpen(false);
        setWorkerMenuOpen(false);
        setUserMenuOpen(false);
        setMobileMenuOpen(false);
      }
    };
  });

```

Nav bar page

```

import React, { useState } from 'react';

/**
 * UserProfile Component
 * Official Civic Profile & Municipal Account Settings Page
 * Strictly adheres to Eco-Civic Green institutional design system.
 */
export default function UserProfile({
  currentUser,
  onUpdateUser,
  onNotification,
  onSwitchTab,
}) {
  const [fullName, setFullName] = useState(currentUser?.full_name || 'Shiva');
  const [email] = useState(currentUser?.email || 'shiva@citizen.nagarmitra.gov.in');
  const [phone, setPhone] = useState(currentUser?.phone || '+91 98765-43210');
  const [ward, setWard] = useState(currentUser?.ward || 'Ward 8 - Ashok Nagar');
  const [address, setAddress] = useState(currentUser?.address || 'Plot 42, Ashok Nagar Main Road, Ward 8');
  const [preferredLanguage, setPreferredLanguage] = useState(currentUser?.preferred_language || currentUser?.language || 't');
  const [notificationPref, setNotificationPref] = useState(true);
  const [smsAlerts, setSmsAlerts] = useState(true);
  const [isEditing, setIsEditing] = useState(false);
  const [savedSuccess, setSavedSuccess] = useState(false);

  const handleSaveProfile = (e) => {
    e.preventDefault();
    const updated = {
      ...currentUser,
      full_name: fullName.trim(),
      phone: phone.trim(),
      ward,
      address: address.trim(),
      preferred_language: preferredLanguage,
      notificationPref,
      smsAlerts,
    };

    if (onUpdateUser) onUpdateUser(updated);
    setIsEditing(false);
    setSavedSuccess(true);
    if (onNotification) {

```

User profile

```

import os
import sys
import uuid
from datetime import datetime
from pathlib import Path
# pyrefly: ignore [missing-import]
from flask import Flask, request, jsonify, Blueprint, send_from_directory, current_app # type: ignore
# pyrefly: ignore [missing-import]
from flask_cors import CORS # type: ignore
# pyrefly: ignore [missing-import]
from werkzeug.utils import secure_filename # type: ignore

# Add project root to sys.path for ai_service imports
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

# pyrefly: ignore [missing-import]
from backend.config import config_by_name, DevelopmentConfig # type: ignore
# pyrefly: ignore [missing-import]
from backend.models import db, User, Department, Worker, Complaint, ComplaintLog, Feedback # type: ignore

try:
    # pyrefly: ignore [missing-import]
    from ai_service.model import classifier as ai_classifier # type: ignore
except ImportError:
    ai_classifier = None

# =====
# Blueprints Initialization
# =====
auth_bp = Blueprint('auth', __name__, url_prefix='/api/auth')
complaints_bp = Blueprint('complaints', __name__, url_prefix='/api/complaints')
manager_bp = Blueprint('manager', __name__, url_prefix='/api/manager')
worker_bp = Blueprint('worker', __name__, url_prefix='/api/worker')
ai_bp = Blueprint('ai', __name__, url_prefix='/api/ai')
feedback_bp = Blueprint('feedback', __name__, url_prefix='/api/feedback')

# -----
# Auth Blueprint Routes (Sign Up & Sign In)
# -----

```

[app.py\(backend\)](#)

```

class User(db.Model):
    """System users: citizens, department managers, field workers, and admins."""
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    full_name = db.Column(db.String(150), nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(255), nullable=False)
    phone = db.Column(db.String(25), nullable=True)
    preferred_language = db.Column(db.String(50), default='English', nullable=True)
    role = db.Column(
        db.Enum('citizen', 'manager', 'worker', 'admin', name='user_roles'),
        default='citizen',
        nullable=False,
        index=True
    )
    is_active = db.Column(db.Boolean, default=True, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    complaints = db.relationship('Complaint', backref='citizen', lazy='dynamic', foreign_keys='Complaint.citizen_id')
    worker_profile = db.relationship('Worker', backref='user', uselist=False, cascade="all, delete-orphan")
    feedbacks = db.relationship('Feedback', backref='citizen', lazy='dynamic')

    def __init__(self, **kwargs):
        for key, value in kwargs.items():
            setattr(self, key, value)

    def set_password(self, password: str):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password: str) -> bool:
        return check_password_hash(self.password_hash, password)

    def to_dict(self) -> dict:
        return {
            'id': self.id,
            'full_name': self.full_name,
            'email': self.email

```

saicharan0806 (3 days ago) Ln 10, Col 1 Spaces: 4 UTF-8 LF {} Python 3.14.7

Model_py

```
"""
CivicSync Backend Configuration
Handles environment variables, database URI generation (MySQL via PyMySQL),
file upload paths, and application security settings.
"""

import os
from pathlib import Path
from dotenv import load_dotenv

env_path = Path(__file__).resolve().parent.parent / '.env'
if env_path.exists():
    load_dotenv(dotenv_path=env_path)
else:
    load_dotenv()

class Config:
    """Base Configuration with Connection Pooling"""
    SECRET_KEY = os.getenv("SECRET_KEY", "civisync-dev-super-secret-key-2026")

    # MySQL Database Configuration
    MYSQL_USER = os.getenv("MYSQL_USER", "root")
    MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "password")
    MYSQL_HOST = os.getenv("MYSQL_HOST", "127.0.0.1")
    MYSQL_PORT = os.getenv("MYSQL_PORT", "3306")
    MYSQL_DB = os.getenv("MYSQL_DB", "civisync_db")

    # Primary SQLAlchemy Database URI using PyMySQL
    DEFAULT_MYSQL_URI = (
        f"mysql+pymysql://{MYSQL_USER}:{MYSQL_PASSWORD}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DB}?charset=utf8mb4"
    )
```

Database Configuration

GEO - TAG PHOTOS

